

DSSMV 2025 (ProjectDroid) - CoverToCover

Vítor Teixeira, Miguel Silva and Lucas Santos

Instituto Superior de Engenharia do Porto,
R. Dr. António Bernardino de Almeida 431, 4249-015 Porto
{1232067,1241131,1241008}@isep.ipp.pt
<https://www.portal.isep.ipp.pt>

Abstract. This report describes the production process of the CoverToCover app. It features diagrams, code snippets, unit tests, and potential improvements. CoverToCover is an app design for maximum privacy whilst reviewing books.

Key words: Java, Android, Gradle, UML, PlantUML, Android Studio, Books, Library, Book Review, Map, Camera, Barcode, ISEP, IPP, LETI, DSSMV, restDB, Google, Google Books, CameraX, MLKit, Google Maps, MPAndroidChart

1 Introduction

Social anxiety has become the main course of life. Any time you leave a like, a dislike, a comment, you run the risk of being judged by your own opinion. As displeased people in a highly critical world, something had to be done. So we made "CoverToCover". Whilst using our application, users can review books at their will and without judgment. It is an Android-only app designed to give users maximum privacy when logging and reviewing books. It runs on Android 10 through 16 devices, using camera, GPS, and internet features. Made using Java and many java coffees.

1.1 Features

The app provides a lot of features to make the UX (user experience) more pleasant.

The following list demonstrates a panoply, but yet only some examples, of the app's features:

- Seeing statistics and some of your books and reviews in the main screen.
- Adding books manually, by barcode scanning, or by searching them.
- Seeing your nearest libraries.
- Adopt a light colored interface, or a dark one.
- Sync your books and reviews on multiple devices.
- ...

2 Domain Model

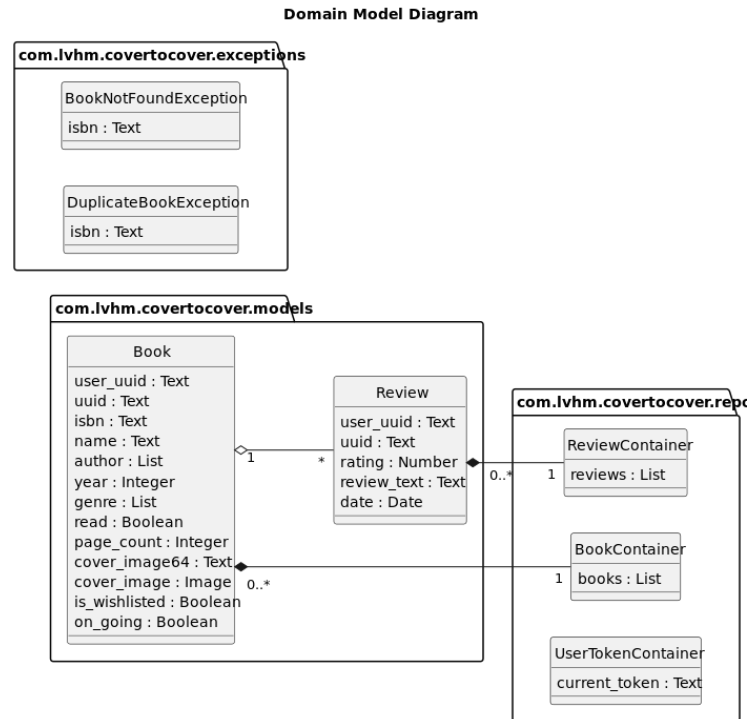


Fig. 1: CoverToCover Domain Model

3 Class Diagram

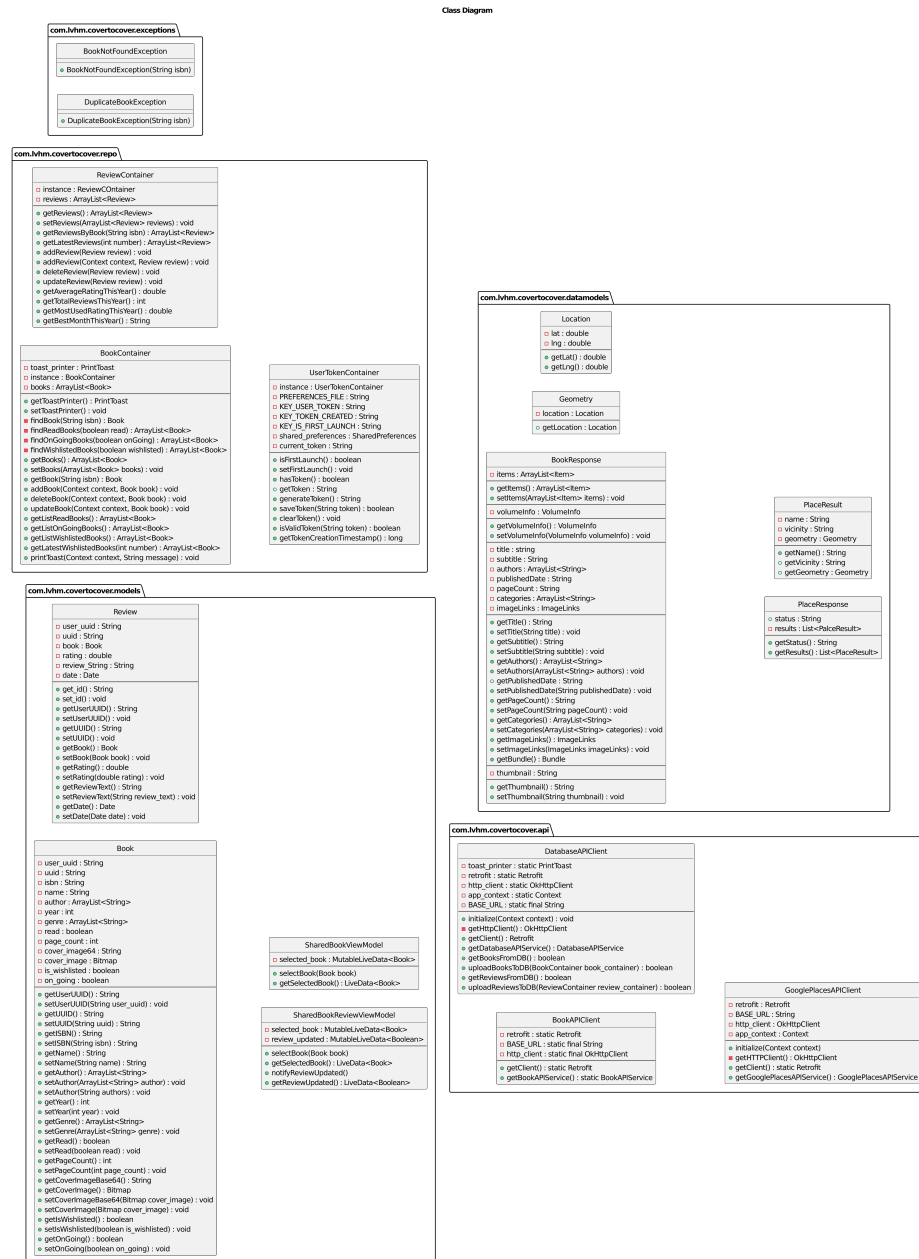


Fig. 2: CoverToCover Class Diagram

4 Non-Functional Requirements

4.1 Usability

- Graphical interface.
- Easy to use user interface.
- Optimal user experience (UX).

4.2 Reliability

- Data Storage must be reliable.

4.3 Performance

- Battery consumption must be minimal.

4.4 Supportability

- The full code is available on the official GitHub page.

4.5 Design Constraints

- Data persistence requires an internet connection (HTTP requests).

4.6 Implementation Constraints

- Java programming language.
- Android versions.

4.7 Interface Constraints

- Android XML components.

5 Functional Requirements

5.1 Main Screen

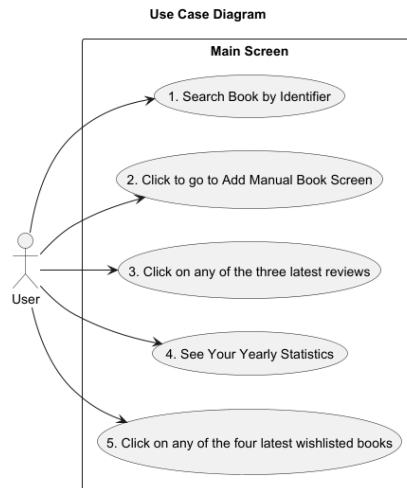


Fig. 3: Main Screen Use Case Diagram

Use Case: Search Book by Identifier (Specification)

Actor	User
Use case name	Search Book by Identifier
Description	The user searches a book, by ISBN, in the search bar
Precondition	The user must be logged in
Post-condition	The user is sent to a new screen
Main flow	1. System displays the Main Screen 2. User presses the search bar 3. System displays an input space 4. User fills the input space with information 5. System will check the API response 6. System displays the Book Screen
Alternative path	5. - API does not answer successfully - Service displays Manual Book Screen
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 1: Use Case: Search Book by Identifier

SSD: UC1 - Search Book by Identifier

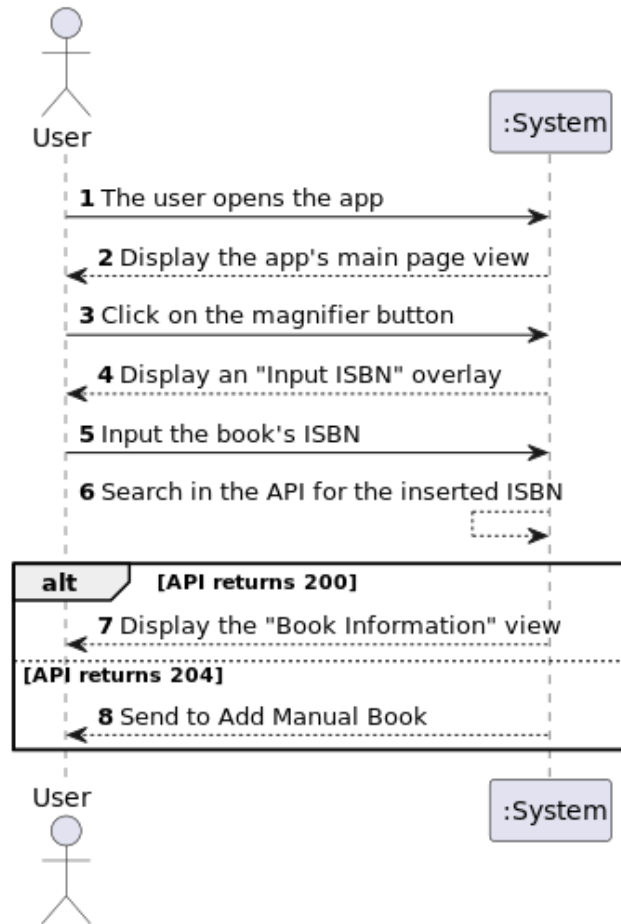


Fig. 4: UC1 System Sequence Diagram

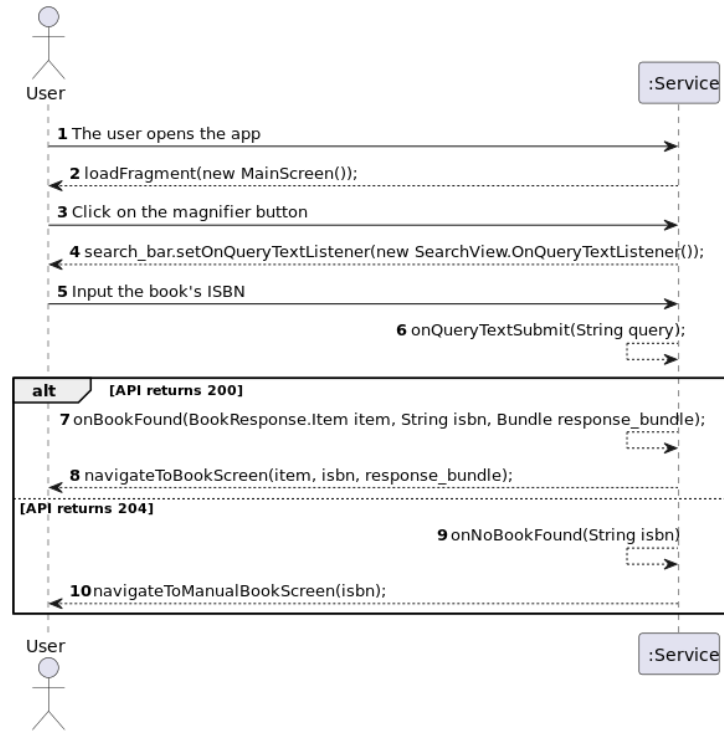
SD: UC1 - Search Book by Identifier

Fig. 5: Main Screen UC1 Sequence Diagram

Use Case: Click to go to Add Manual Book Screen (Specification)

Actor	User
Use case name	Click to go to Add Manual Book Screen
Description	The user clicks the "Add Manual Book" button and is sent to the Manual Book Screen
Precondition	The user must be logged in
Post-condition	The user is sent to a new screen
Main flow	1. System displays the Main Screen 2. User presses the "Add Manual Book" button 3. System displays the Manual Book Screen
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 2: Use Case: Click to go to Add Manual Book Screen

SSD: UC2 - Click to go to Add Manual Book Screen

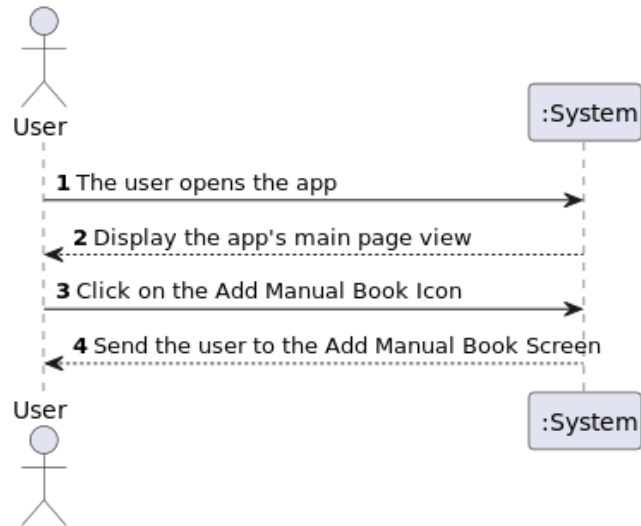


Fig. 6: UC2 System Sequence Diagram

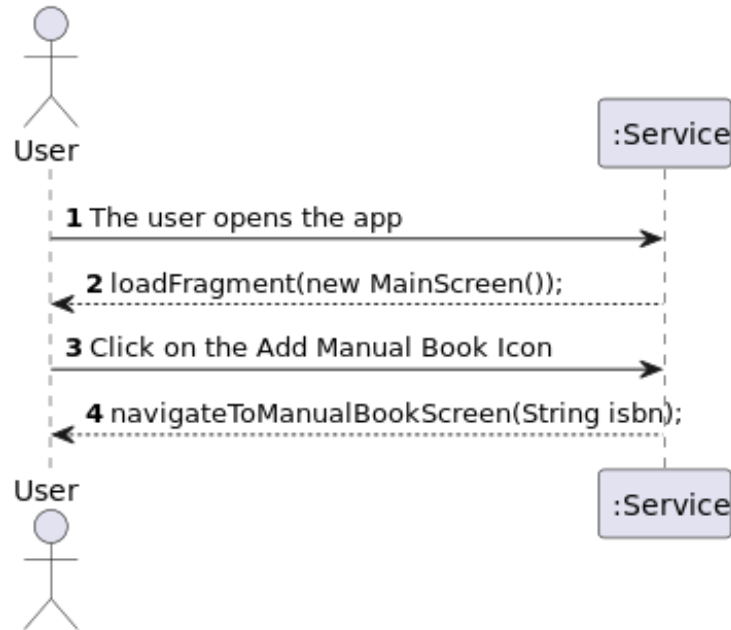
SD: UC2 - Click to go to Add Manual Book Screen

Fig. 7: Main Screen UC2 Sequence Diagram

Use Case: Click on any of the three latest reviews (Specification)

Actor	User
Use case name	Click on any of the three latest reviews
Description	The user clicks on any of the latest reviews and is sent to the respective Book Screen
Precondition	The user must be logged in
Post-condition	The user is sent to a new screen
Main flow	1. System displays the Main Screen 2. User presses on any of the latest reviews 3. System displays the respective Book Screen
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 3: Use Case: Click on any of the three latest reviews

SSD: UC3 - Click on any of the three latest reviews

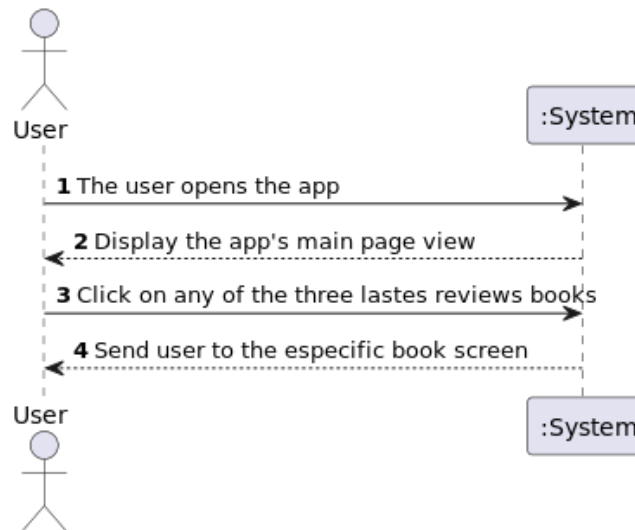


Fig. 8: UC3 System Sequence Diagram

SD: UC3 - Click on any of the three latest reviews

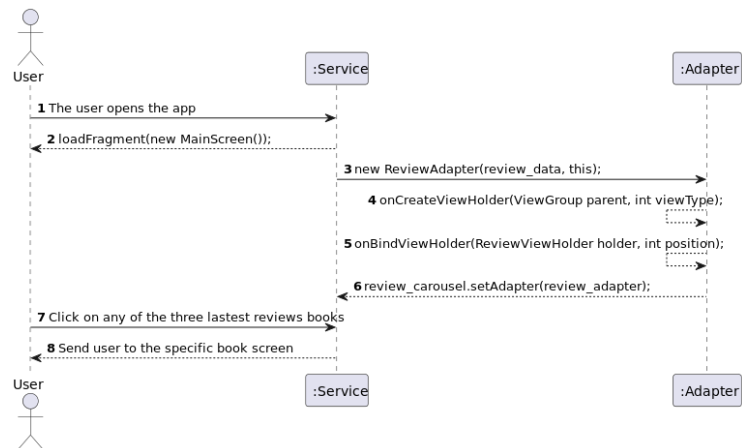


Fig. 9: Main Screen UC3 Sequence Diagram

Use Case: See Your Yearly Statistics (Specification)

Actor	User
Use case name	See Your Yearly Statistics
Description	The user sees the space on the Main Screen that indicates the yearly statistics
Precondition	The user must be logged in
Post-condition	The user knows his/her yearly statistics
Main flow	1. System displays the Main Screen 2. System displays the yearly statistics
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 4: Use Case: See Your Yearly Statistics

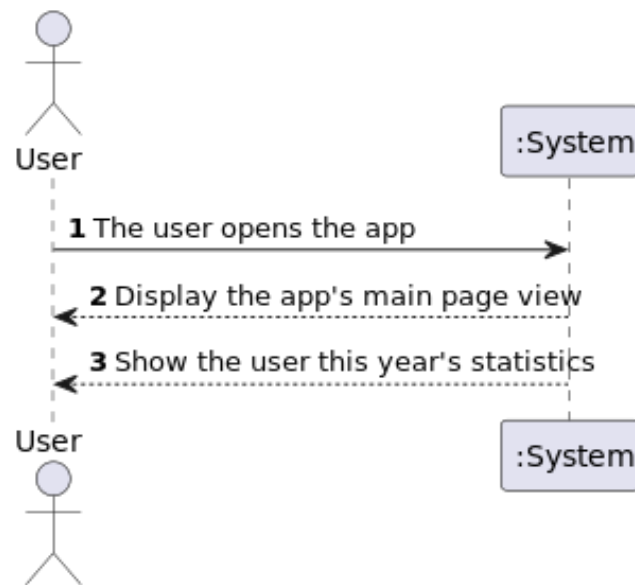
SSD: UC4 - See Your Yearly Statistics

Fig. 10: UC4 System Sequence Diagram

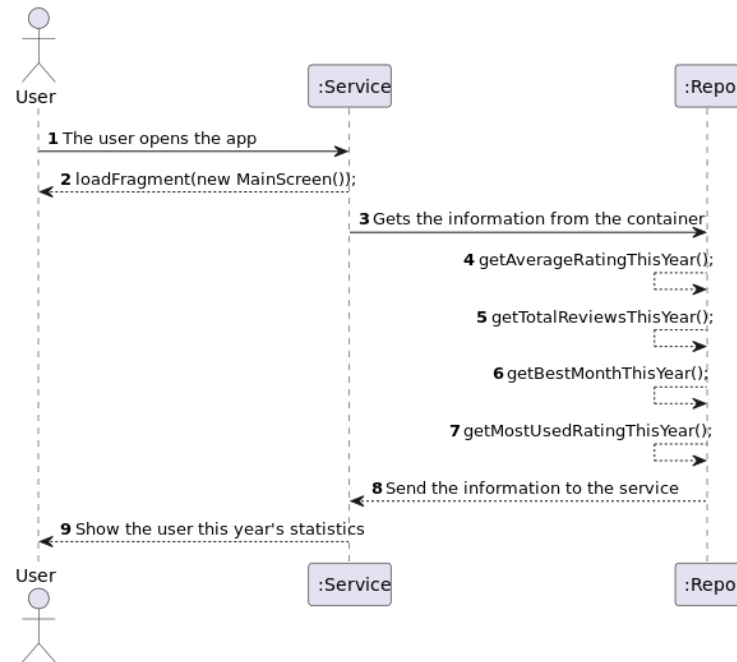
SD: UC4 - See Your Yearly Statistics

Fig. 11: Main Screen UC4 Sequence Diagram

Use Case: Click on any of the four latest wishlisted books (Specification)

Actor	User
Use case name	Click on any of the four latest wishlisted books
Description	The user clicks on any of the four latest wishlisted books and is sent to the respective Book Screen
Precondition	The user must be logged in
Post-condition	The user is sent to a new screen
Main flow	1. System displays the Main Screen 2. User presses on any of the latest wishlisted books 3. System displays the respective Book Screen
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 5: Use Case: Click on any of the four latest wishlisted books

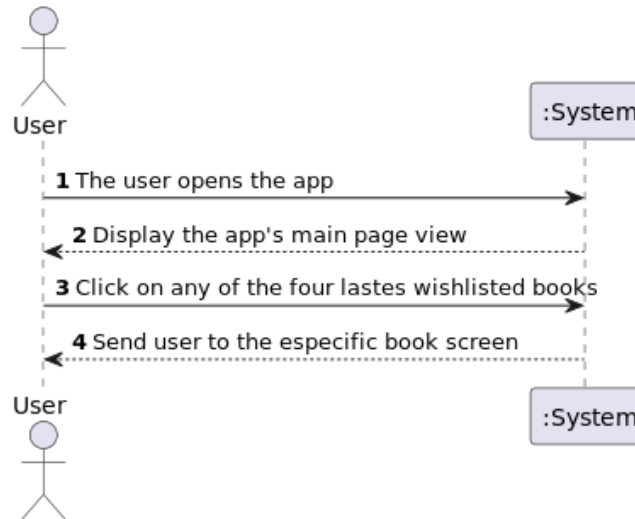
SSD: UC5 - Click on any of the four latest wishlisted books

Fig. 12: UC5 System Sequence Diagram

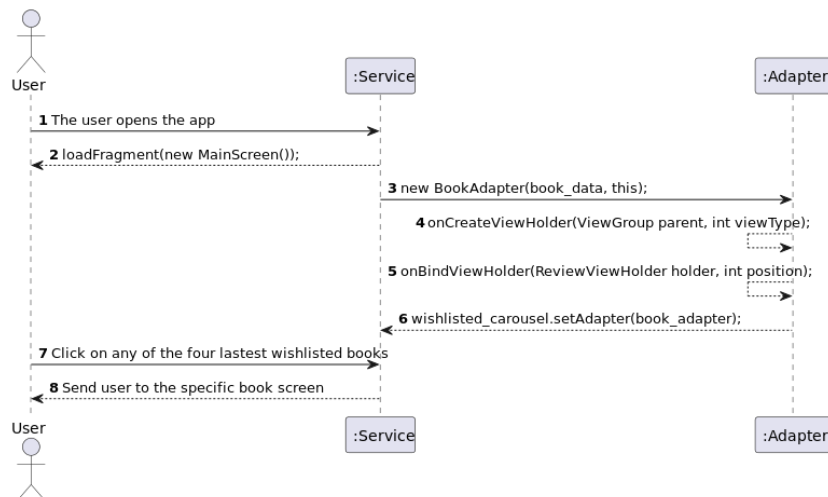
SD: UC5 - Click on any of the four latest wishlisted books

Fig. 13: Main Screen UC5 Sequence Diagram

5.2 Profile Screen

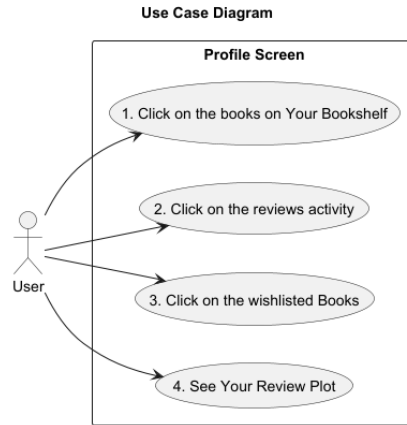


Fig. 14: Profile Screen Use Case

Use Case: Click on the books on Your Bookshelf (Specification)

Actor	User
Use case name	Click on the books on Your Bookshelf
Description	The user clicks on any of the books on Your Bookshelf and is sent to the respective Book Screen
Precondition	The user must be logged in
Post-condition	The user is sent to a new screen
Main flow	1. System displays the Profile Screen 2. User presses on any of the books 3. System displays the respective Book Screen
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 6: Use Case: Click on the books on Your Bookshelf

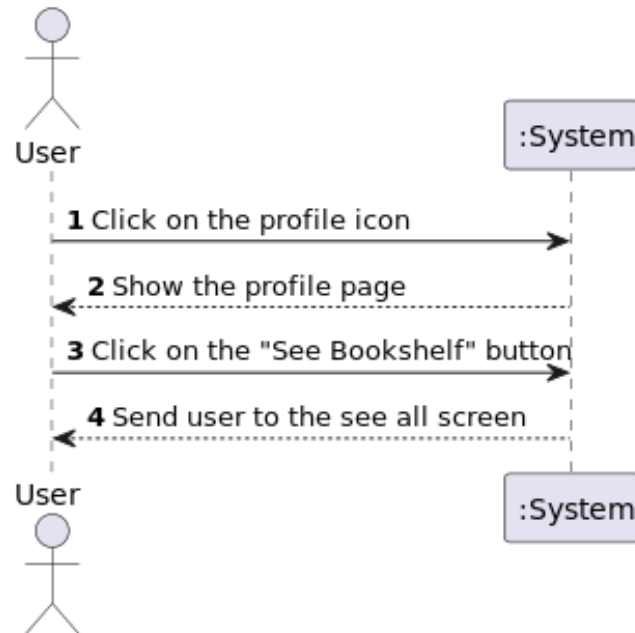
SSD: UC1 - Click on the books on Your Bookshelf

Fig. 15: UC1 System Sequence Diagram

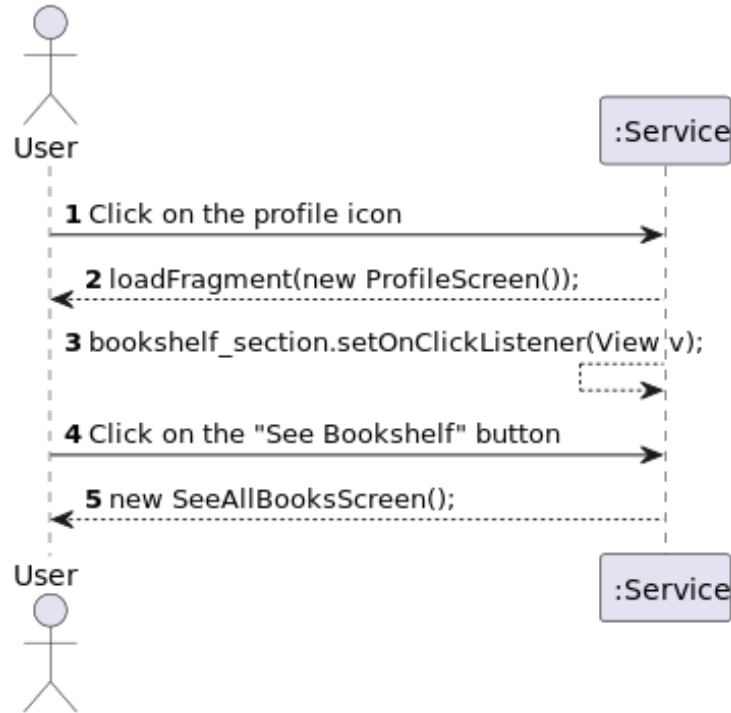
SD: UC1 - Click on the books on Your Bookshelf

Fig. 16: Profile Screen UC1 Sequence Diagram

Use Case: Click on the reviews activity (Specification)

Actor	User
Use case name	Click on the reviews activity
Description	The user clicks on any of the reviews on Review Activity and is sent to the respective Book Screen
Precondition	The user must be logged in
Post-condition	The user is sent to a new screen
Main flow	1. System displays the Profile Screen 2. User presses on any of the reviews 3. System displays the respective Book Screen
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 7: Use Case: Click on the reviews activity

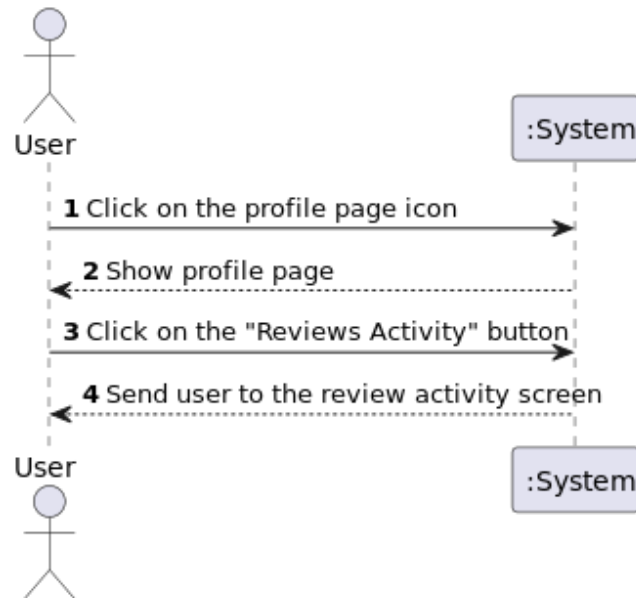
SSD: UC2 - Click on the reviews activity

Fig. 17: UC2 System Sequence Diagram

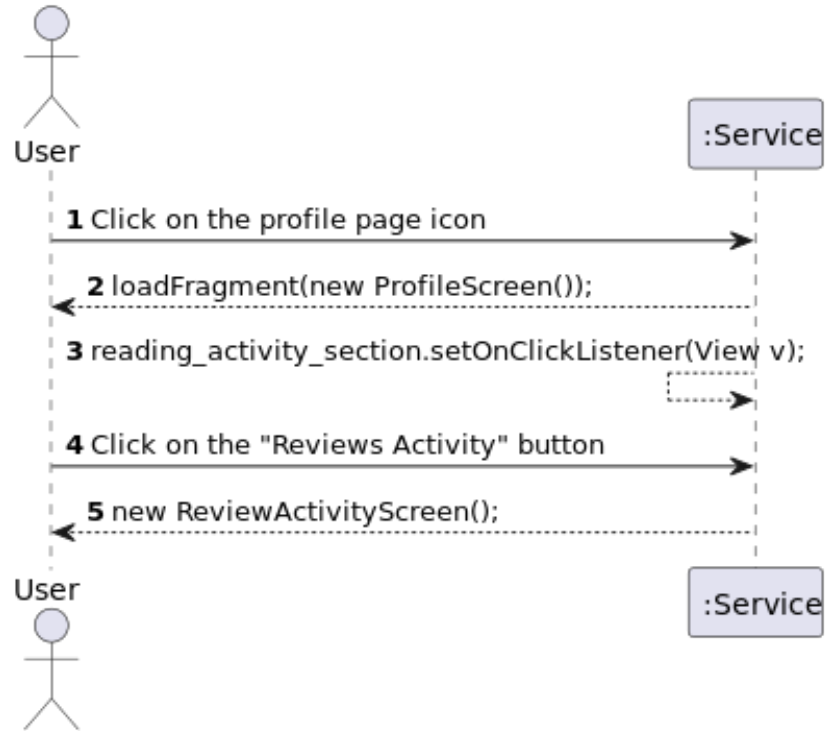
SD: UC2 - Click on the reviews activity

Fig. 18: Profile Screen UC2 Sequence Diagram

Use Case: Click on the wishlisted books (Specification)

Actor	User
Use case name	Click on the wishlisted books
Description	The user clicks on any of the wishlisted books on Your Wish-listed Books and is sent to the respective Book Screen
Precondition	The user must be logged in
Post-condition	The user is sent to a new screen
Main flow	1. System displays the Profile Screen 2. User presses on any of the wishlisted books 3. System displays the respective Book Screen
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 8: Use Case: Click on the wishlisted books

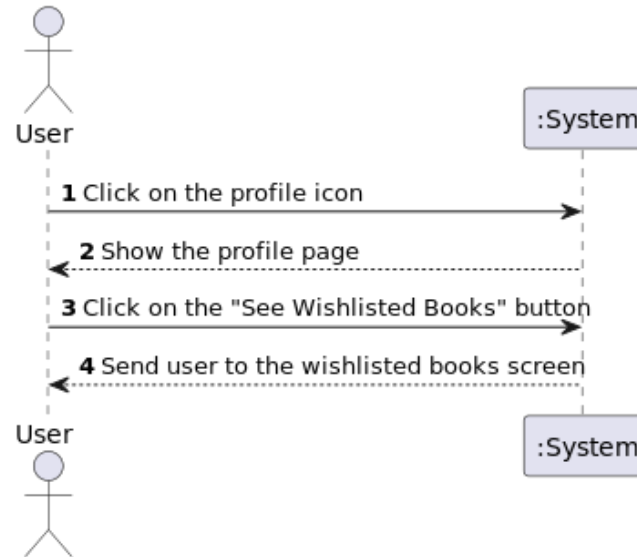
SSD: UC3 - Click on the wishlisted books

Fig. 19: UC3 System Sequence Diagram

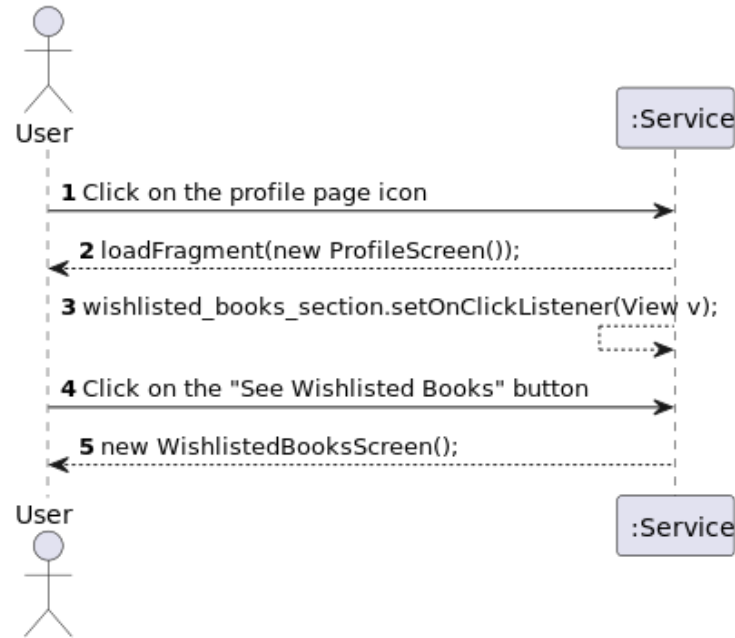
SD: UC3 - Click on the wishlisted books

Fig. 20: Profile Screen UC3 Sequence Diagram

Use Case: See your review plot (Specification)

Actor	User
Use case name	See your review plot
Description	The user see the plot counting all of his/her review ratings
Precondition	The user must be logged in
Post-condition	The user becomes aware of the review plot
Main flow	1. System displays the Profile Screen 2. System displays the review plot
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 9: Use Case: See your review plot

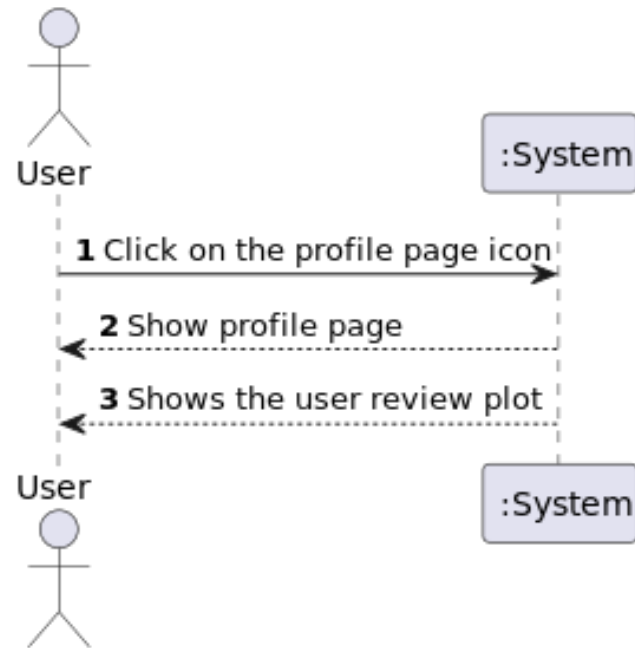
SSD: UC4 - See your review plot

Fig. 21: UC4 System Sequence Diagram

SD: UC4 - See your review plot

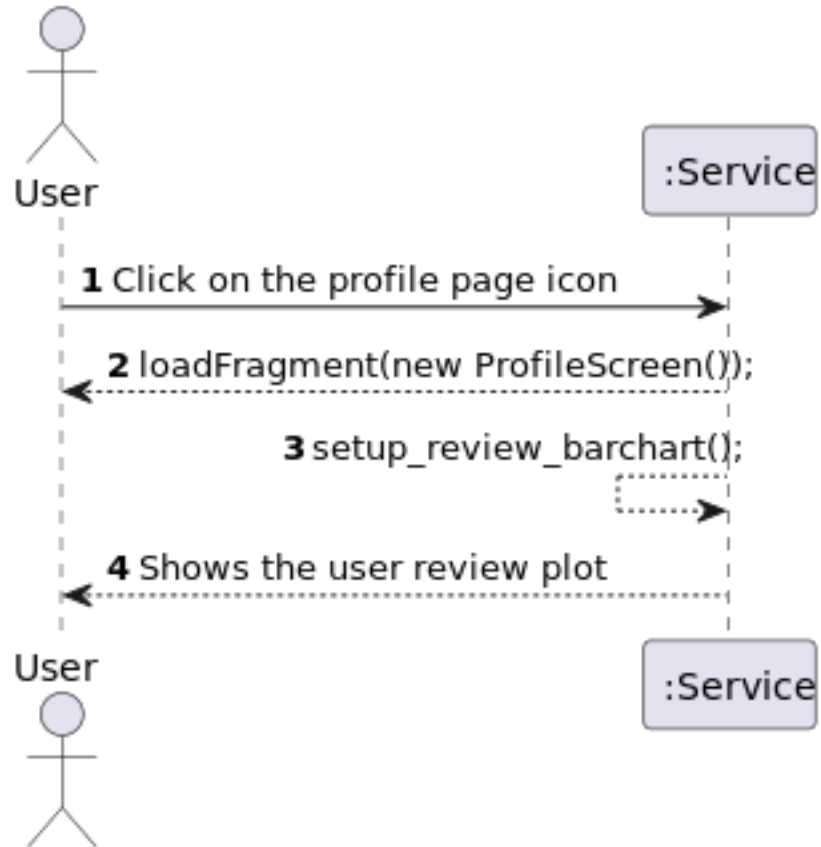


Fig. 22: Profile Screen UC5 Sequence Diagram

5.3 Camera Screen

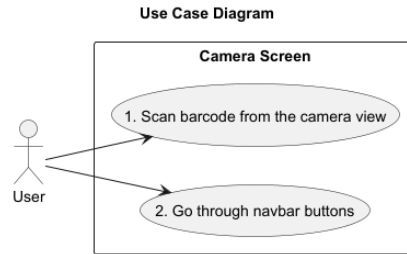


Fig. 23: Camera Screen Use Case

Use Case: Scan barcode from the camera view (Specification)

Actor	User
Use case name	Scan barcode from the camera view
Description	The user clicks on the shutter button and is sent, depending on the API response, either to a Book Screen or a Manual Book Screen
Precondition	The user must be logged in
Post-condition	The user is sent to a new screen
Main flow	1. System displays the Camera Screen 2. User presses the shutter button 3. System checks the API response 4. System displays the respective Book Screen
Alternative path	3. - API response is different than a successful one - System displays the Manual Book Screen
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 10: Use Case: Scan barcode from the camera view

SSD: UC1 - Scan barcode from the camera view

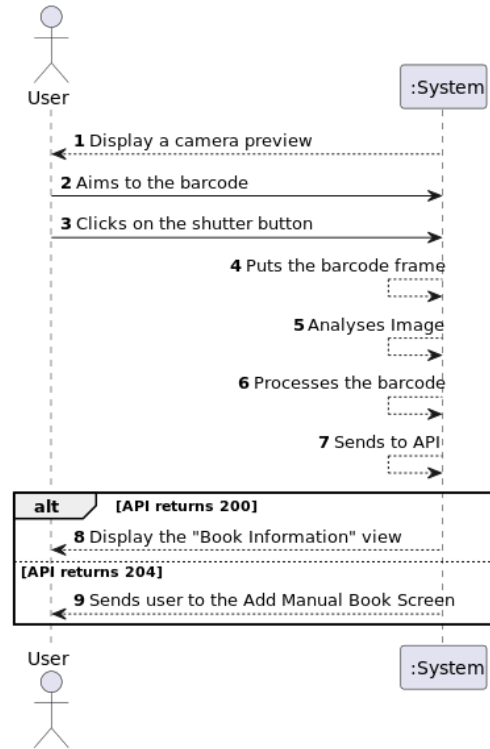


Fig. 24: UC1 System Sequence Diagram

SD: UC1 - Scan barcode from the camera view

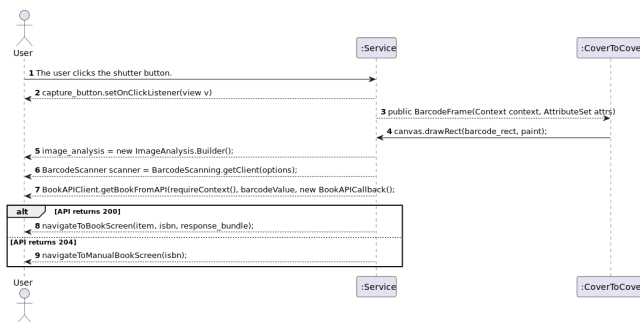


Fig. 25: Camera Screen UC1 Sequence Diagram

5.4 Map Screen

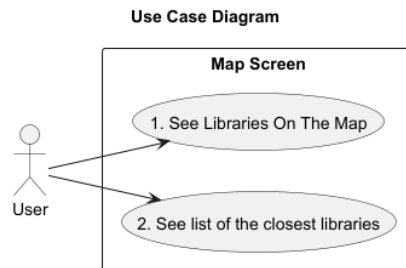


Fig. 26: Map Use Case

Use Case: See Libraries On The Map (Specification)

Actor	User
Use case name	See Libraries On The Map
Description	The user sees the pins, in the map, pointing to libraries near the user
Precondition	The user must be logged in
Post-condition	The user is now aware that such libraries exist
Main flow	1. System displays the Map Screen 2. System displays the location on the map
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 11: Use Case: See Libraries On The Map

SSD: UC1 - See Libraries On The Map

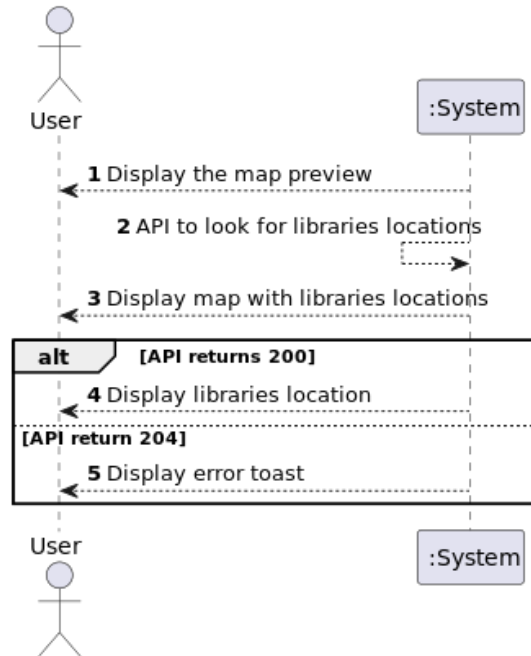


Fig. 27: UC1 System Sequence Diagram

SD: UC1 - See Libraries On The Map

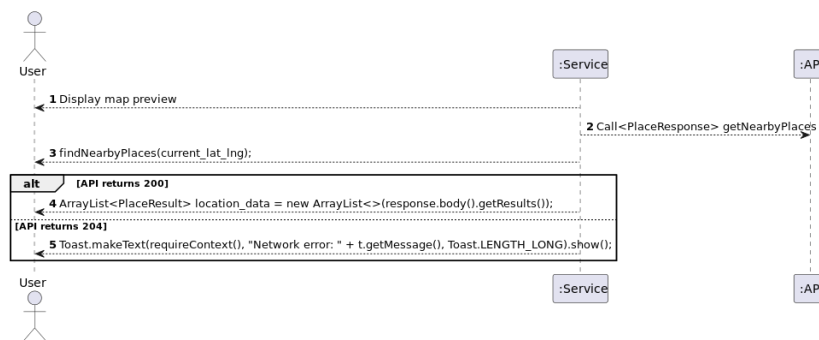


Fig. 28: Map Screen UC1 Sequence Diagram

Use Case: See list of the closest libraries (Specification)

Actor	User
Use case name	See list of the closest libraries
Description	The user sees the list of libraries near the him/her
Precondition	The user must be logged in
Post-condition	The user is now aware that such libraries exist
Main flow	1. System displays the Map Screen 2. System displays the list of locations
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 12: Use Case: See list of the closest libraries

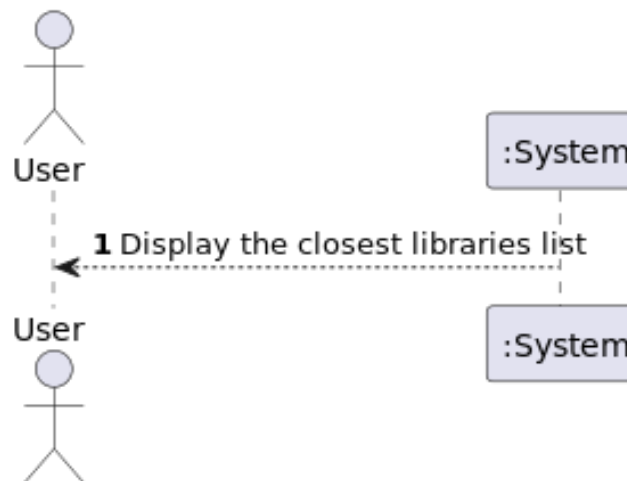
SSD: UC2 - See list of the closest libraries

Fig. 29: UC2 System Sequence Diagram

SD: UC2 - See list of the closest libraries

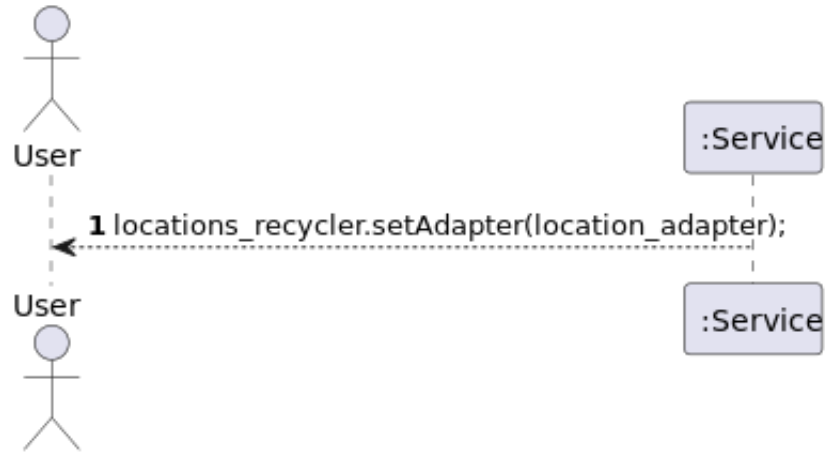


Fig. 30: Map Screen UC2 Sequence Diagram

5.5 Settings Screen

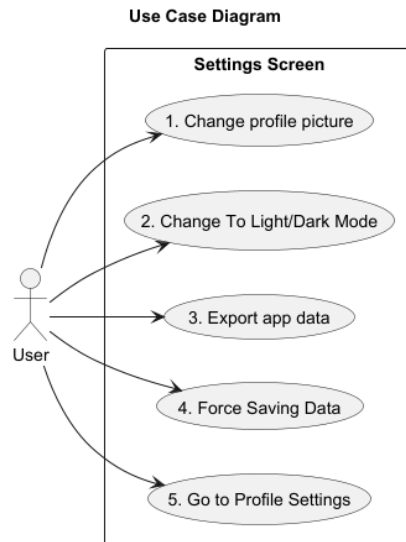


Fig. 31: Settings Screen Use Case Diagram

Use Case: Change profile picture (Specification)

Actor	User
Use case name	Change profile picture
Description	The user clicks the profile picture placeholder
Precondition	The user must be logged in
Post-condition	The new profile picture is saved
Main flow	1. System displays the settings screen 2. User clicks on profile picture place holder 3. System opens the gallery 4. User chooses his new profile picture 5. System saves the new profile picture
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 13: Use Case: Change profile picture

SSD: UC1 - Change profile picture

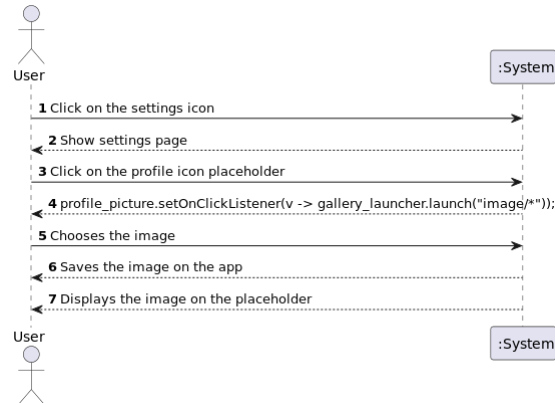


Fig. 32: UC1 System Sequence Diagram

SD: UC1 - Change profile picture

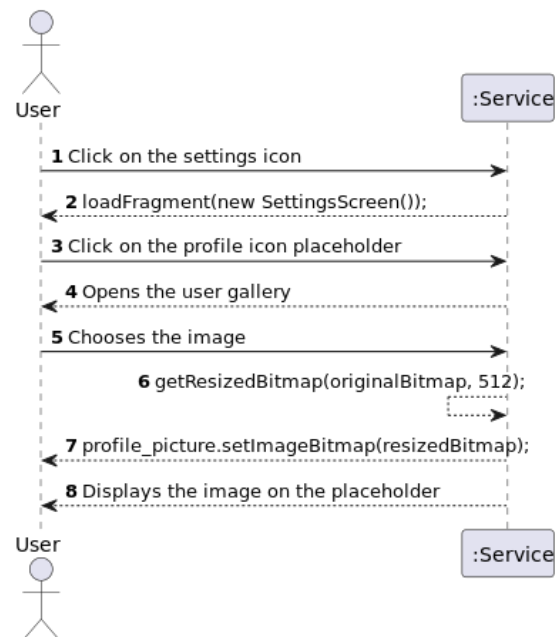


Fig. 33: Settings Screen UC1 Sequence Diagram

Use Case: Change To Light/Dark Mode (Specification)

Actor	User
Use case name	Change To Light/Dark mode
Description	The user clicks the change theme dropdown menu
Precondition	The user must be logged in
Post-condition	The new theme is applied
Main flow	1. System displays the settings screen 2. User clicks on change theme dropdown menu 3. User chooses his new theme 4. System saves the new theme
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 14: Use Case: Change To Light/Dark Mode

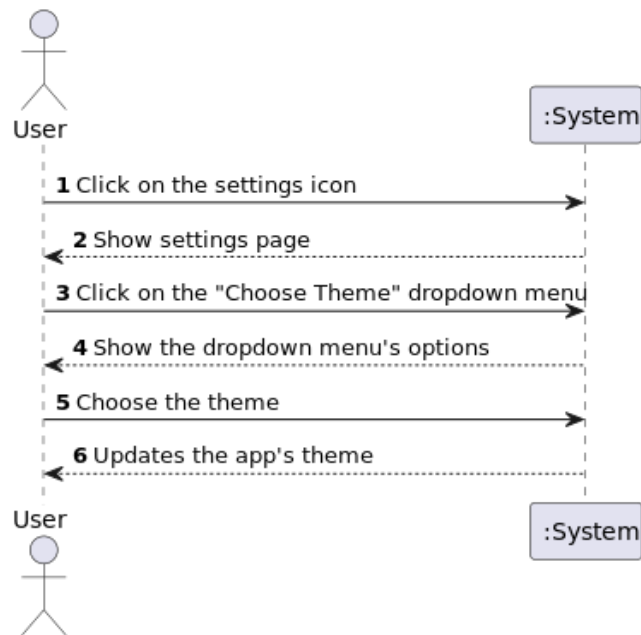
SSD: UC2 - Change To Light/Dark Mode

Fig. 34: UC2 System Sequence Diagram

SD: UC2 - Change To Light/Dark Mode

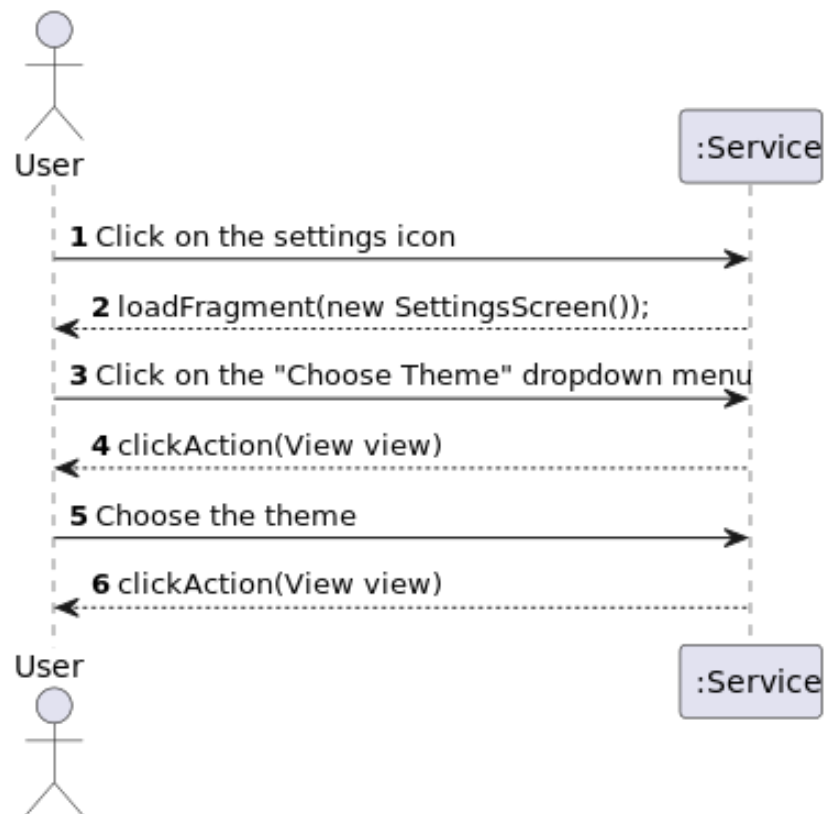


Fig. 35: Settings Screen UC2 Sequence Diagram

Use Case: Export app data (Specification)

Actor	User
Use case name	Export app data
Description	The user clicks on the export app data dropdown menu
Precondition	The user must be logged in
Post-condition	App data is now exported
Main flow	1. System displays the settings screen 2. User clicks on export app data dropdown menu 3. User chooses which format he is going to export the app data 4. System sends the data to the export choice
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 15: Use Case: Change profile picture

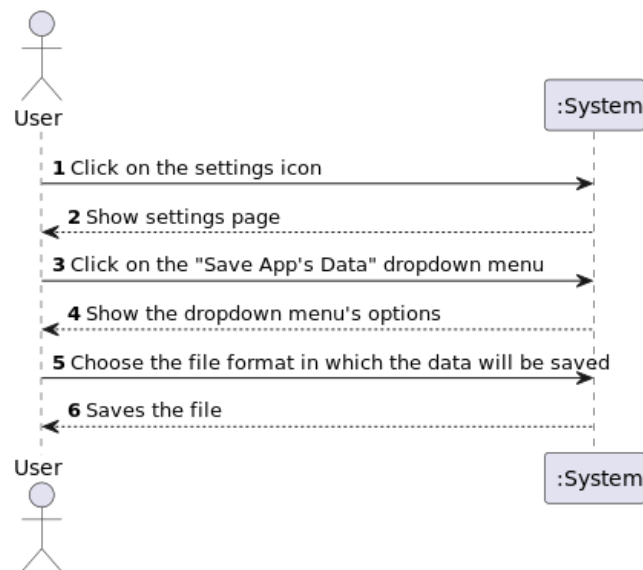
SSD: UC3 - Export app data

Fig. 36: UC3 System Sequence Diagram

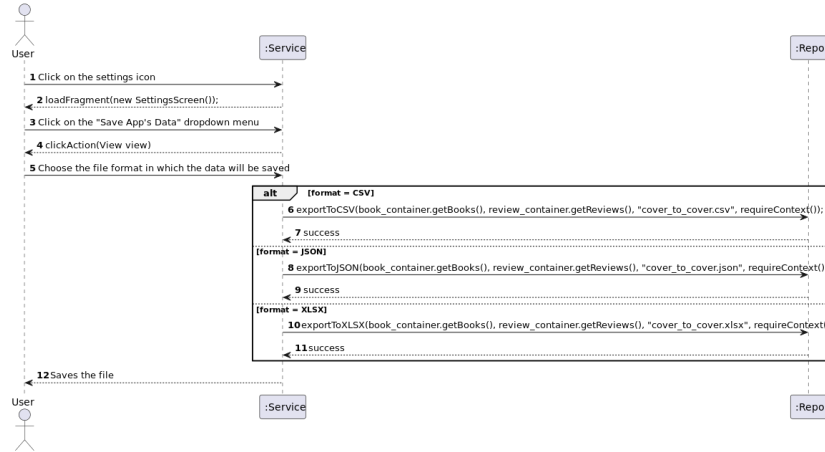
SD: UC3 - Export app data

Fig. 37: Settings Screen UC3 Sequence Diagram

Use Case: Force Saving Data (Specification)

Actor	User
Use case name	Force Saving Data
Description	The user clicks on the force save data button
Precondition	The user must be logged in
Post-condition	The user data is sent to the data base
Main flow	1. System displays the settings screen 2. User clicks on force save data 3. System saves the user data into the data base 4. User chooses his new profile picture 5. System saves the new profile picture
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 16: Use Case: Change profile picture

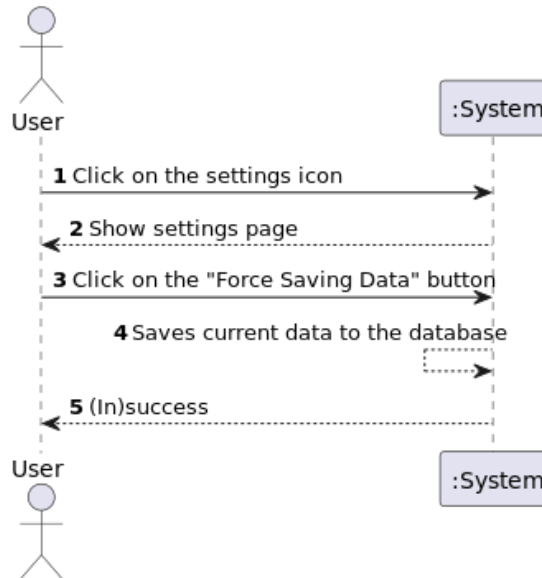
SSD: UC4 - Force Saving Data

Fig. 38: UC4 System Sequence Diagram

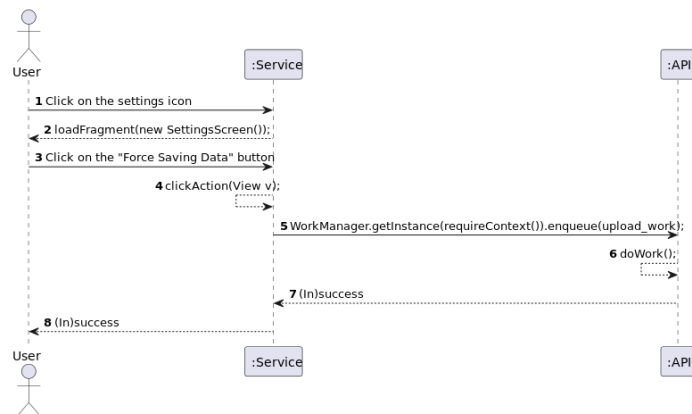
SD: UC4 - Force Saving Data

Fig. 39: Settings Screen UC4 Sequence Diagram

Use Case: Go to Profile Settings (Specification)

Actor	User
Use case name	Go to Profile Settings
Description	The user clicks on the profile settings button
Precondition	The user must be logged in
Post-condition	The user is sent to profile settings screen
Main flow	1. System displays the settings screen 2. User clicks on profile settings button 3. System sends user to the profile settings screen
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 17: Use Case: Go to Profile Settings

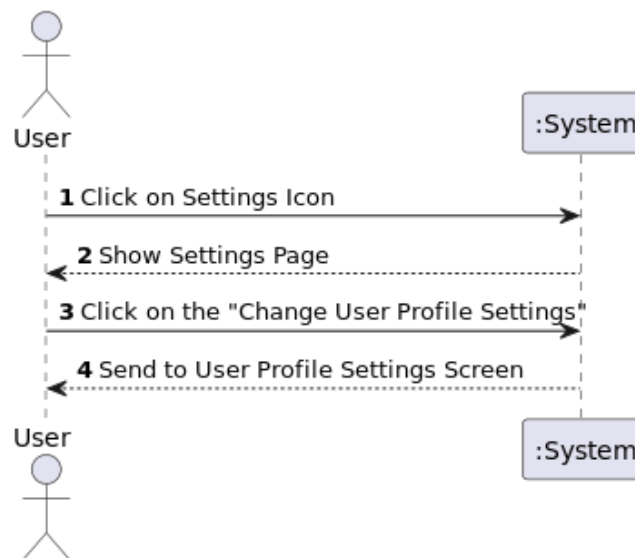
SSD: UC5 - Go to Profile Settings

Fig. 40: UC5 System Sequence Diagram

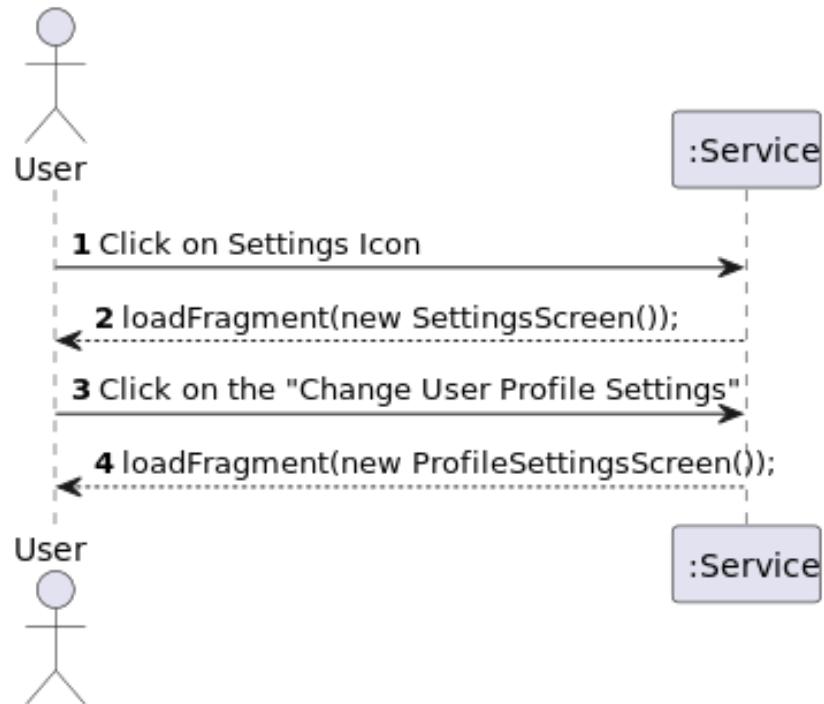
SD: UC5 - Go to Profile Settings

Fig. 41: Settings Screen UC5 Sequence Diagram

5.6 Profile Settings Screen

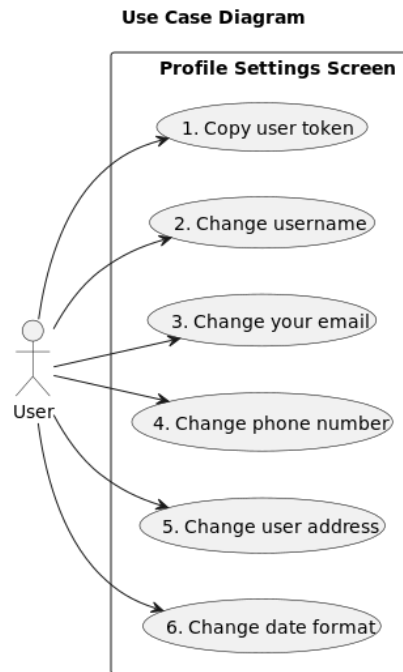


Fig. 42: Profile Settings Screen Use Case Diagram

Use Case: Copy user token (Specification)

Actor	User
Use case name	Copy user token
Description	The user clicks on an icon to copy his token
Precondition	The user must be logged in
Post-condition	Token is copied to the user's clipboard
Main flow	1. System displays the profile settings screen 2. User clicks on the copy token button 3. System sends the token to the user's clipboard
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 18: Use Case: Copy user token

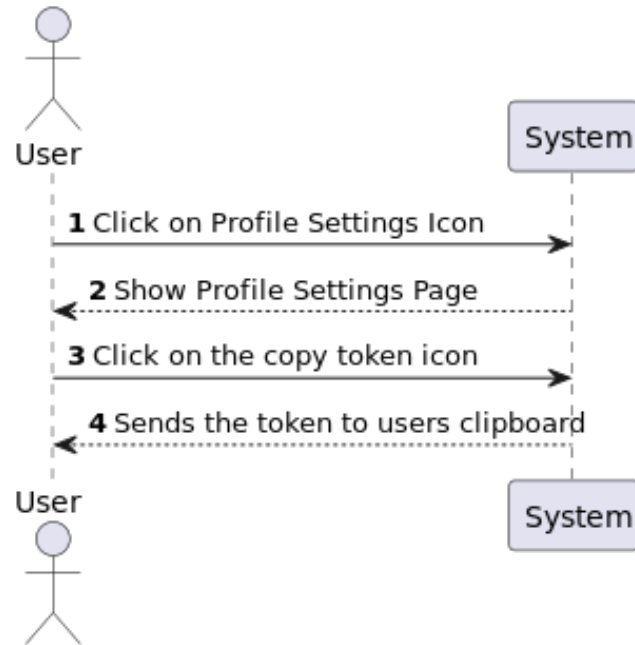
SSD: UC1 - Copy user token

Fig. 43: UC1 System Sequence Diagram

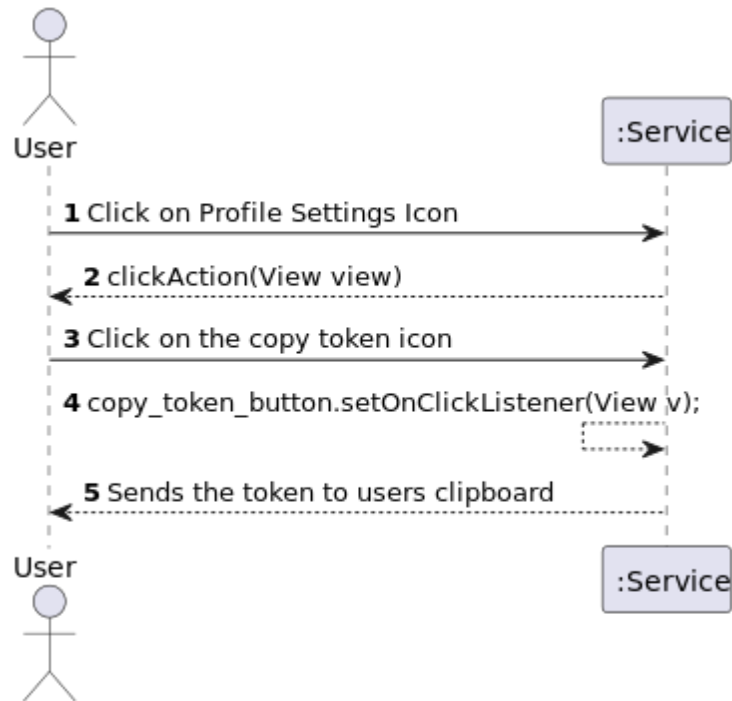
SD: UC1 - Copy user token

Fig. 44: UC1 Sequence Diagram

Use Case: Change username (Specification)

Actor	User
Use case name	Change username
Description	The user clicks the username editable text
Precondition	The user must be logged in
Post-condition	The new username is saved
Main flow	1. System displays the profile settings screen 2. User clicks on username editable text 3. System saves the new username
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 19: Use Case: Change username

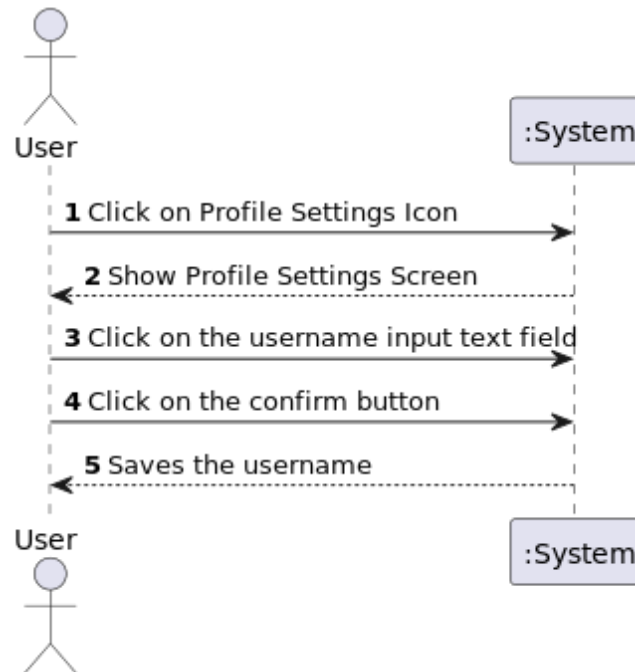
SSD: UC2 - Change username

Fig. 45: UC2 System Sequence Diagram

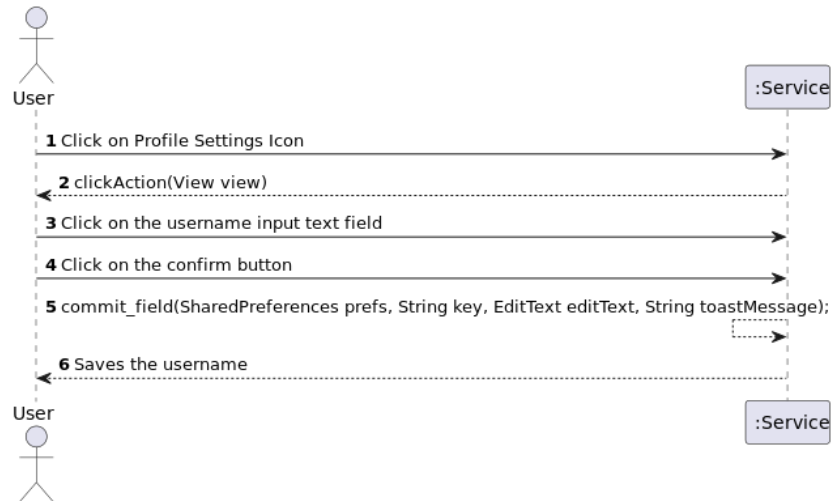
SD: UC2 - Change username

Fig. 46: UC2 Sequence Diagram

Use Case: Change your email (Specification)

Actor	User
Use case name	Change email
Description	The user clicks the email editable text
Precondition	The user must be logged in
Post-condition	The new email is saved
Main flow	1. System displays the profile settings screen 2. User clicks on email editable text 3. System saves the new email
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 20: Use Case: Change email

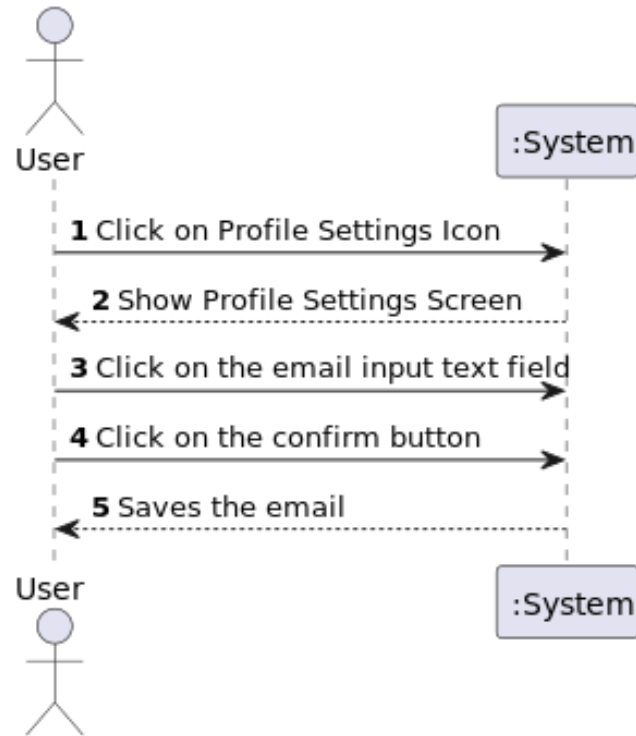
SSD: UC3 - Change your email

Fig. 47: UC3 System Sequence Diagram

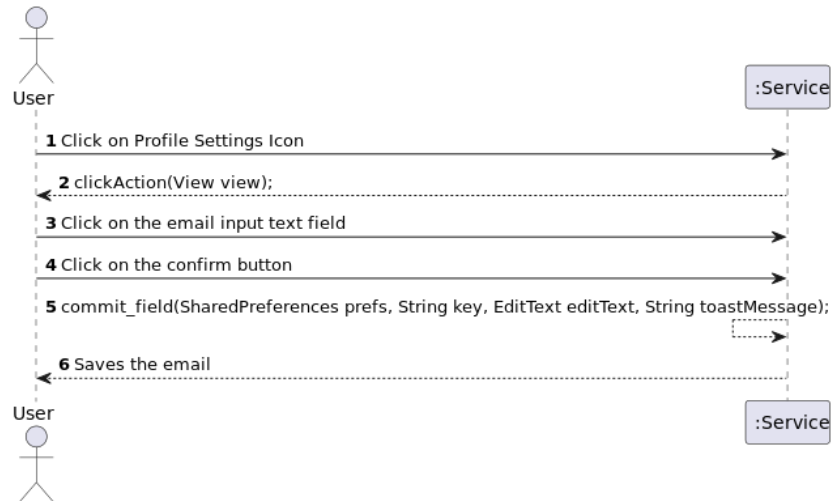
SD: UC3 - Change your email

Fig. 48: UC3 Sequence Diagram

Use Case: Change phone number (Specification)

Actor	User
Use case name	Change phone number
Description	The user clicks the phone number editable text
Precondition	The user must be logged in
Post-condition	The new phone number is saved
Main flow	1. System displays the profile settings screen 2. User clicks on phone number editable text 3. System saves the new phone number
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 21: Use Case: Change phone number

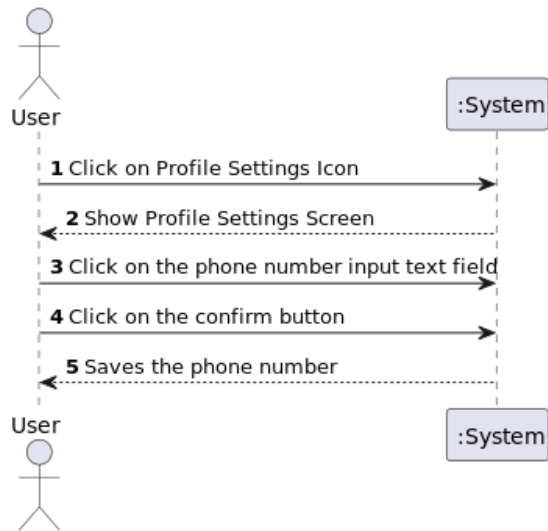
SSD: UC4 - Change phone number

Fig. 49: UC4 System Sequence Diagram

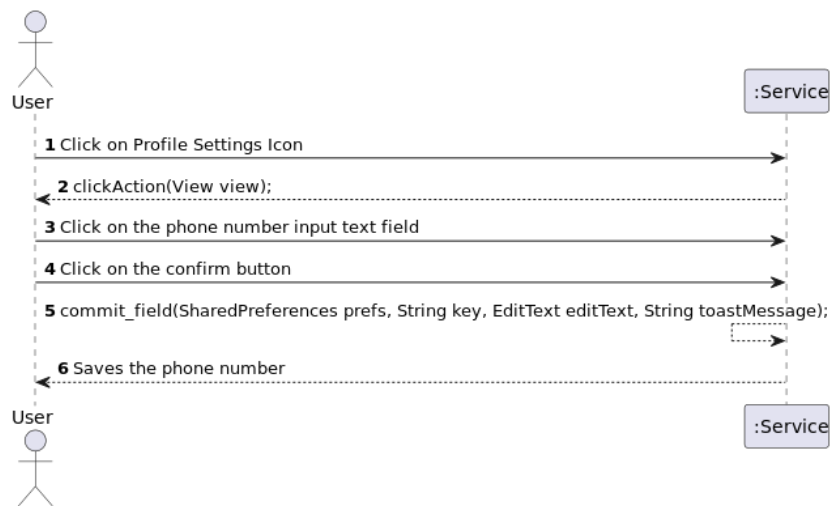
SD: UC4 - Change phone number

Fig. 50: UC4 Sequence Diagram

Use Case: Change user address (Specification)

Actor	User
Use case name	Change address
Description	The user clicks the address editable text
Precondition	The user must be logged in
Post-condition	The new address is saved
Main flow	1. System displays the profile settings screen 2. User clicks on address editable text 3. System saves the new address
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 22: Use Case: Change address

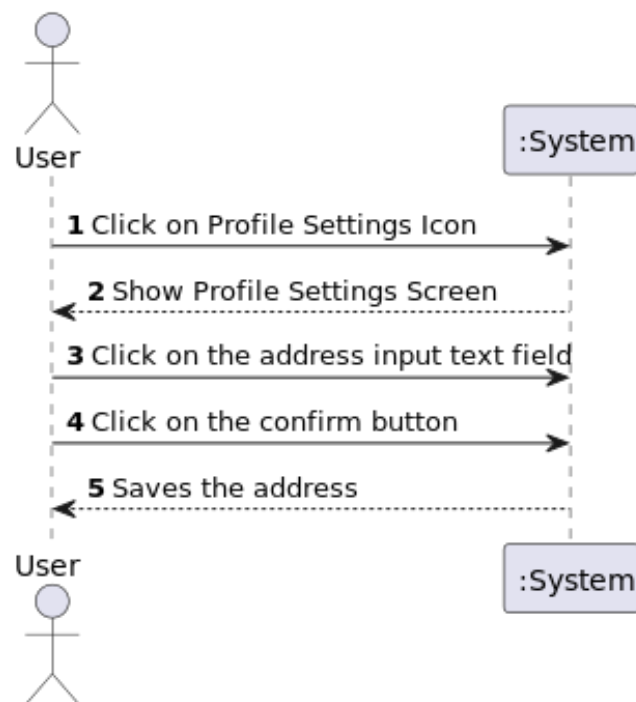
SSD: UC5 - Change user address

Fig. 51: UC5 System Sequence Diagram

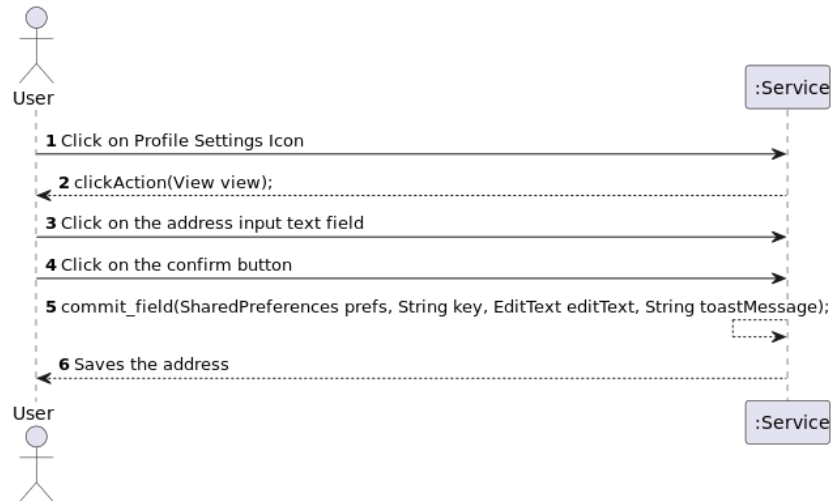
SD: UC5 - Change user address

Fig. 52: UC5 Sequence Diagram

Use Case: Change date format (Specification)

Actor	User
Use case name	Change date format
Description	The user clicks the date format dropdown menu
Precondition	The user must be logged in
Post-condition	The new data format is saved
Main flow	1. System displays the profile settings screen 2. User clicks on username dropdown menu 3. User chooses the new data format 3. System saves the new data format
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 23: Use Case: Change username

SSD: UC6 - Change date format

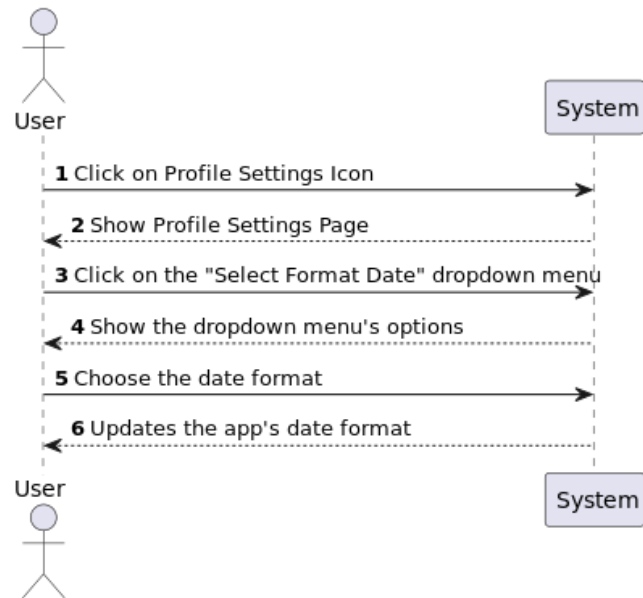


Fig. 53: UC6 System Sequence Diagram

5.7 Manual Book Screen

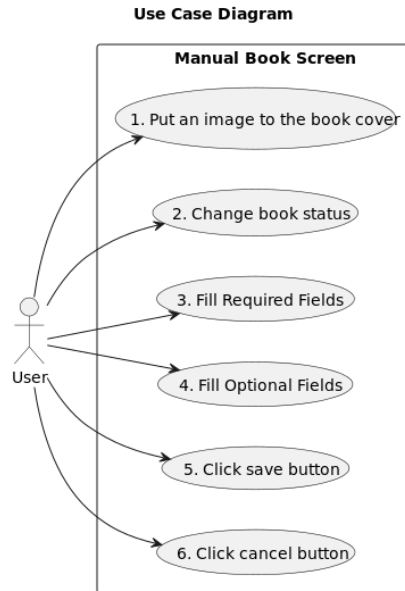


Fig. 54: Manual Book Screen Screen Use Case Diagram

Use Case: Put an image to the book cover (Specification)

Actor	User
Use case name	Put an image to the book cover
Description	The user clicks on the book cover placeholder and chooses a new cover
Precondition	The user must be logged in
Post-condition	Book is updated with a new cover
Main flow	1. System displays the manual book screen 2. User clicks on the book cover placeholder 3. Systems opens the gallery 4. User chooses the new cover 5. System updates the book cover with a new cover.
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 24: Use Case: Put an image to the book cover

SSD: UC1 - Put an image to the book cover

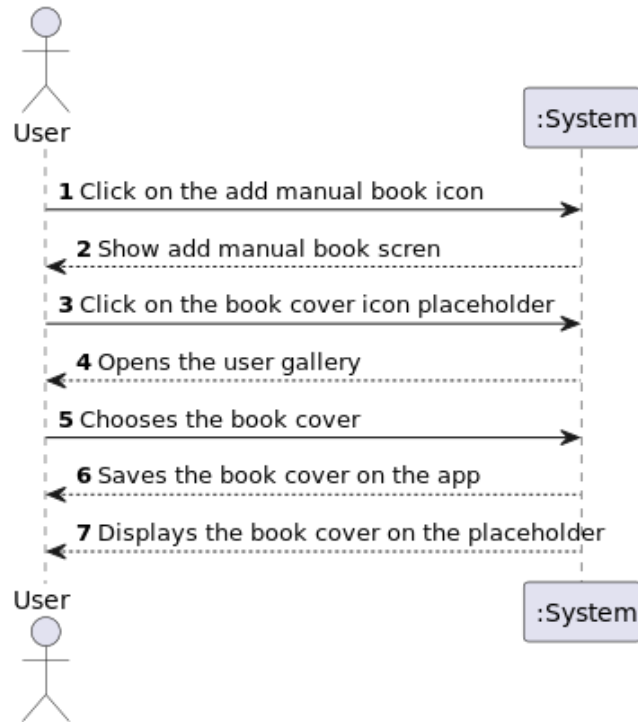


Fig. 55: UC1 System Sequence Diagram

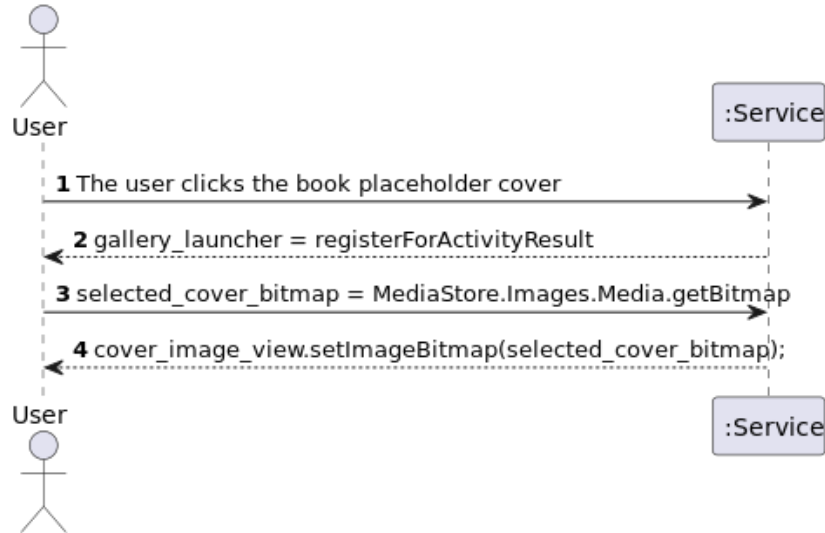
SD: UC1 - Put an image to the book cover

Fig. 56: UC1 Sequence Diagram

Use Case: Change book status (Specification)

Actor	User
Use case name	Change book reading status
Description	The user clicks on the status button to choose his current status on that book
Precondition	The user must be logged in
Post-condition	System saves the new status
Main flow	1. System displays the manual book screen 2. User clicks on the status button 3. User chooses his current status on that book 4. System saves the current status on that book
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 25: Use Case: Change book reading status

SSD: UC2 - Change book status

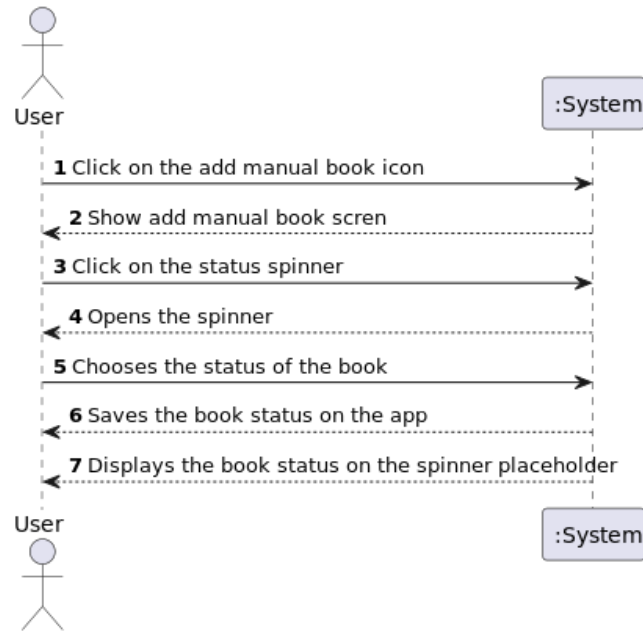


Fig. 57: UC2 System Sequence Diagram

SD: UC2 - Change book status

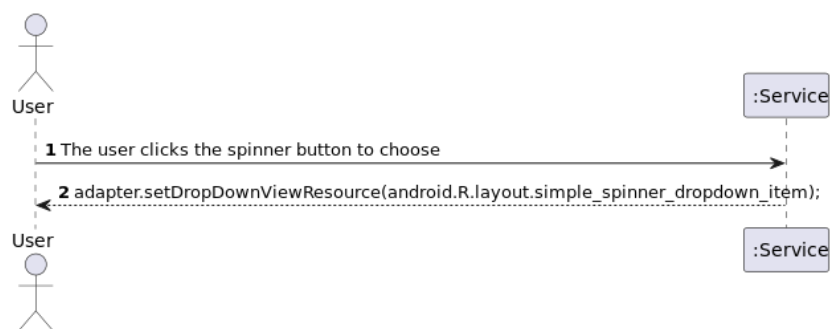


Fig. 58: UC2 Sequence Diagram

Use Case: Fill Required Fields (Specification)

Actor	User
Use case name	Fill Required Fields
Description	The user fills the required fields
Precondition	The user must be logged in
Post-condition	Book is updated with the required fields
Main flow	1. System displays the manual book screen 2. User clicks on the editable text (Required Fields) 3. User writes on the required fields 4. System saves the required fields on that book
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 26: Use Case: Fill Required Fields

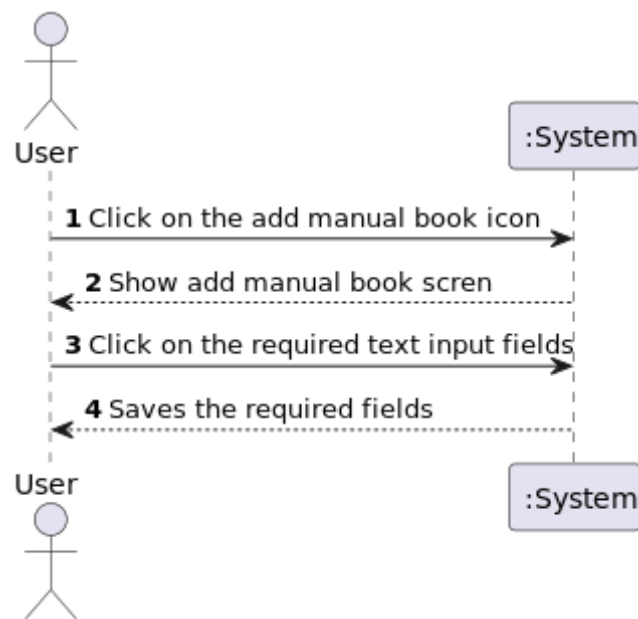
SSD: UC3 - Fill Required Fields

Fig. 59: UC3 System Sequence Diagram

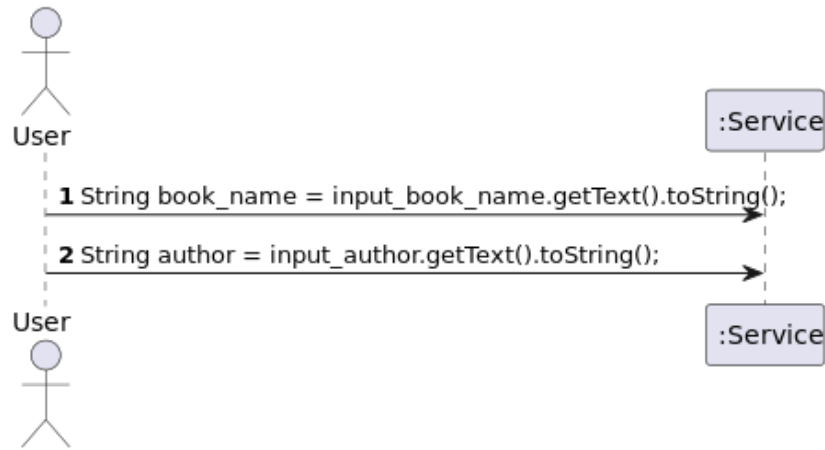
SD: UC3 - Fill Required Fields

Fig. 60: UC3 Sequence Diagram

Use Case: Fill Optional Fields (Specification)

Actor	User
Use case name	Fill Optional Fields
Description	The user fills the optional fields
Precondition	The user must be logged in
Post-condition	Book is updated with the optional fields
Main flow	1. System displays the manual book screen 2. User clicks on the editable text (Optional Fields) 3. User writes on the optional fields 4. System saves the optional fields on that book
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 27: Use Case: Fill Optional Fields

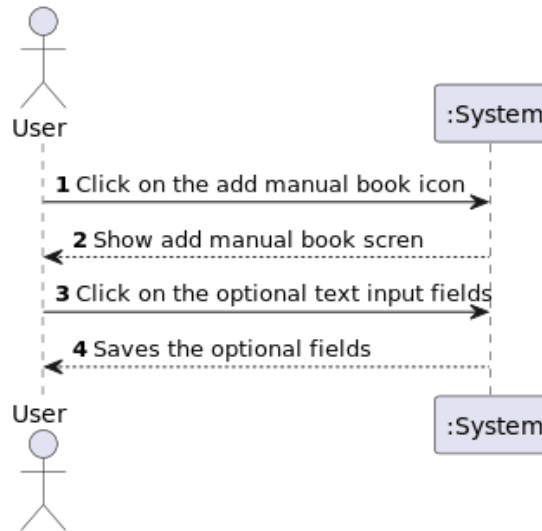
SSD: UC4 - Fill Optional Fields

Fig. 61: UC4 System Sequence Diagram

SD: UC4 - Fill Optional Fields

Fig. 62: UC4 Sequence Diagram

Use Case: Click save button (Specification)

Actor	User
Use case name	Click save button
Description	The user clicks on save button to save the book
Precondition	The user must be logged in
Post-condition	Book is updated in the system
Main flow	1. System displays the manual book screen 2. User clicks on the save button 3. System adds the book
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 28: Use Case: Click save button

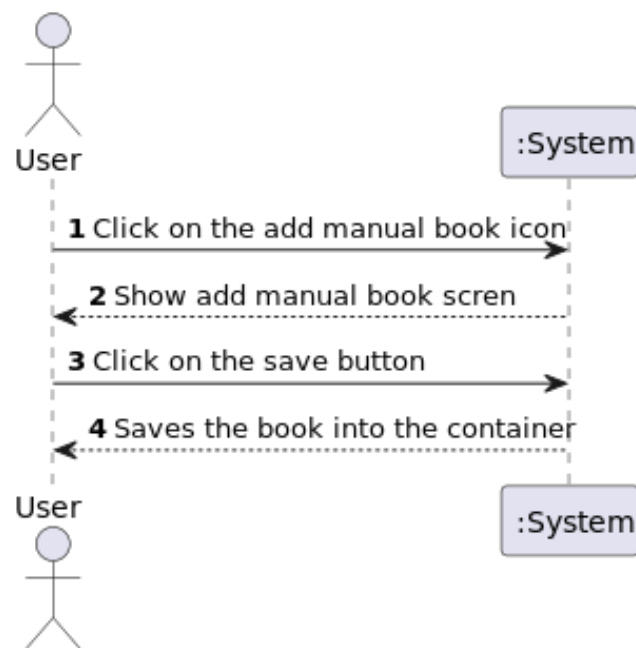
SSD: UC5 - Click save button

Fig. 63: UC5 System Sequence Diagram

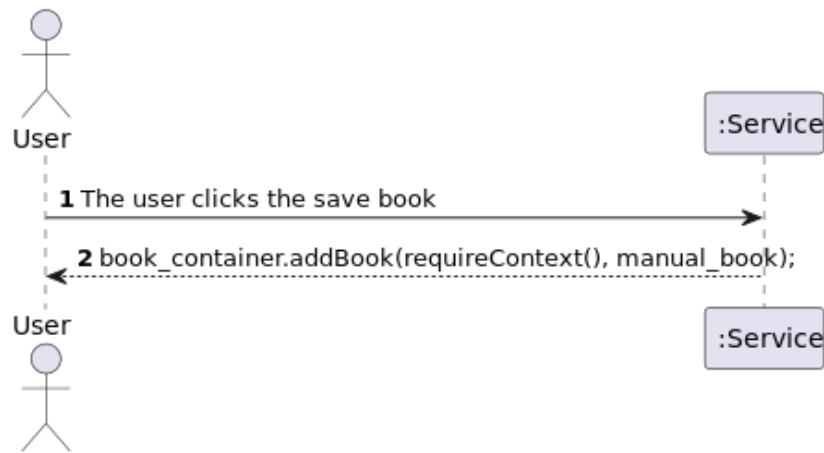
SD: UC5 - Click save button

Fig. 64: UC5 Sequence Diagram

Use Case: Click cancel button (Specification)

Actor	User
Use case name	Click cancel button
Description	The user clicks on cancel button
Precondition	The user must be logged in
Post-condition	The operation is canceled and the user goes to the back page
Main flow	1. System displays the manual book screen 2. User clicks on the cancel button 3. System cancels the operation and sends user to the back page
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 29: Use Case: Fill Required Fields

SSD: UC6 - Click cancel button

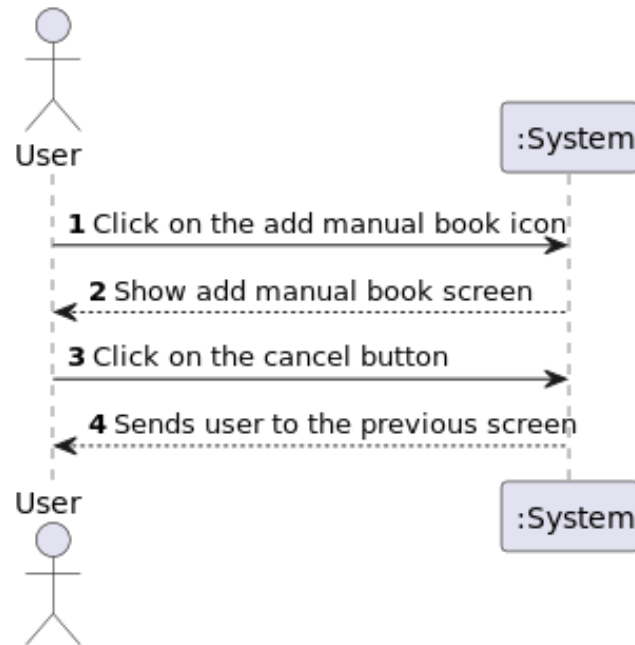


Fig. 65: UC6 System Sequence Diagram

5.8 Book Screen

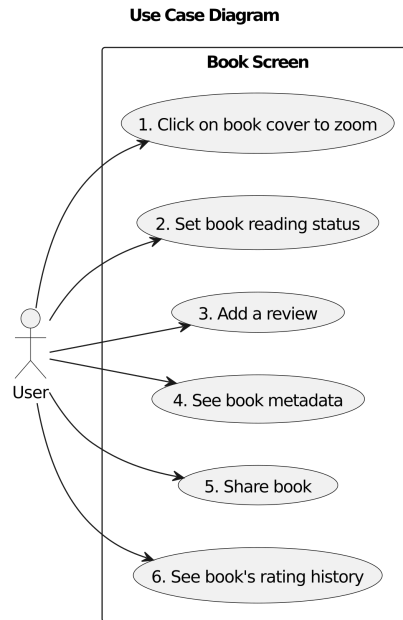


Fig. 66: Book Screen Use Case

Use Case: Click on book cover to zoom (Specification)

Actor	User
Use case name	Click on book cover to zoom
Description	The user clicks on the book cover to open an bigger image of the cover
Precondition	The user must be logged in
Post-condition	User sees a bigger book cover
Main flow	1. System displays the book screen 2. User clicks on the book cover 3. Systems opens an bigger book cover (overlay)
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 30: Use Case: Click on book cover to zoom

SSD: UC1 - Click on book cover to zoom

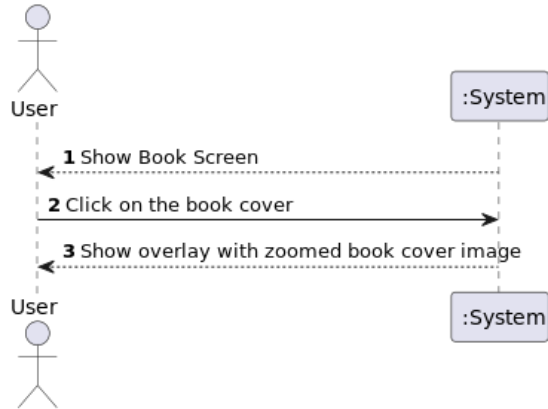


Fig. 67: UC1 System Sequence Diagram

SD: UC1 - Click on book cover to zoom

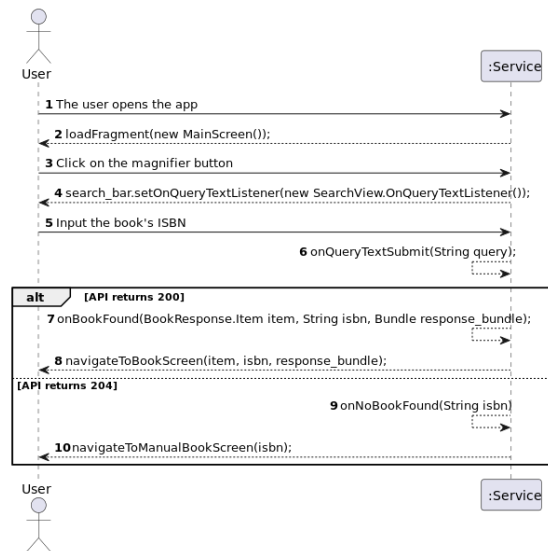


Fig. 68: Book Screen UC1 Sequence Diagram

Use Case: Set book reading status (Specification)

Actor	User
Use case name	Set book reading status
Description	The user clicks on the status button to choose his current status on that book
Precondition	The user must be logged in
Post-condition	System saves the new status
Main flow	1. System displays the book screen 2. User clicks on the status button 3. User chooses his current status on that book 4. System saves the current status on that book
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 31: Use Case: Set book reading status

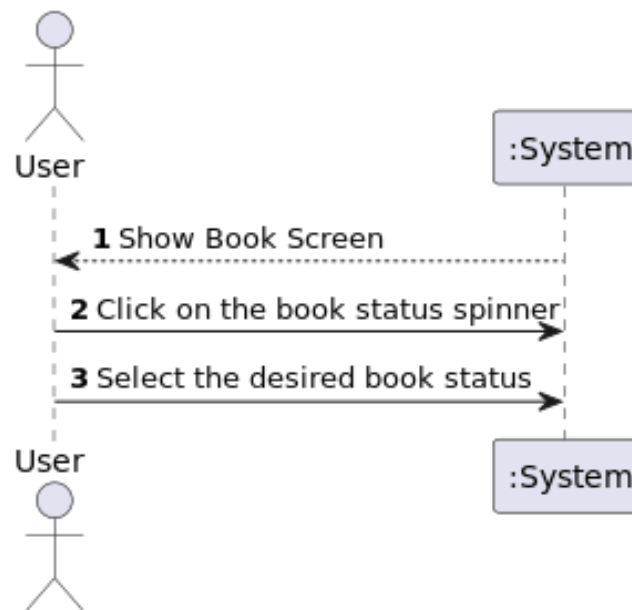
SSD: UC2 - Set book reading status

Fig. 69: UC2 System Sequence Diagram

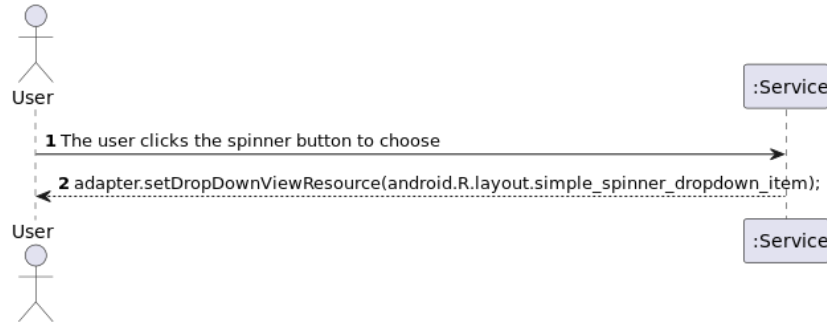
SD: UC2 - Set book reading status

Fig. 70: Book Screen UC2 Sequence Diagram

Use Case: Add a review (Specification)

Actor	User
Use case name	Add a review
Description	The user clicks on the review button and makes an review for a book
Precondition	The user must be logged in
Post-condition	System saves the review for that book
Main flow	1. System displays the book screen 2. User clicks on the review button 3. User chooses his review for the book (rating bar) 4. User writes his review 5. System saves the review for that book 6. System shows the book's review on the review history
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 32: Use Case: Add a review

SSD: UC3 - Add a review

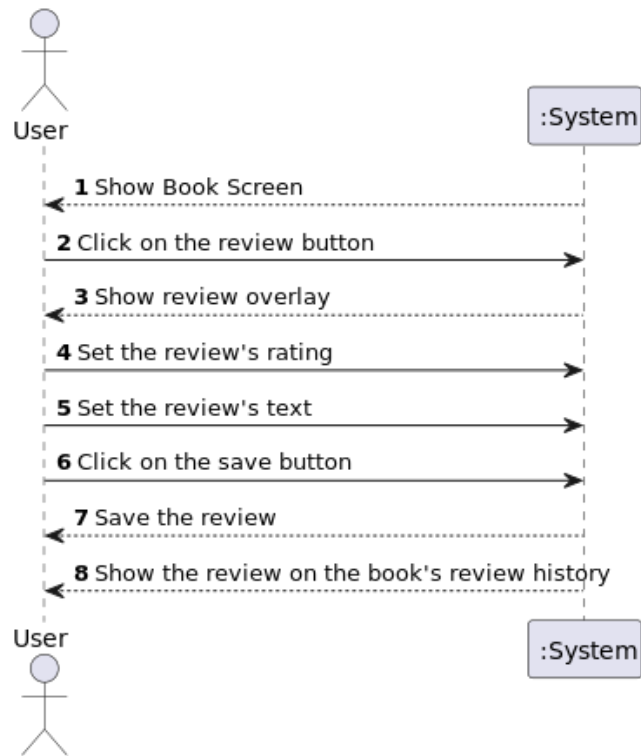


Fig. 71: UC3 System Sequence Diagram

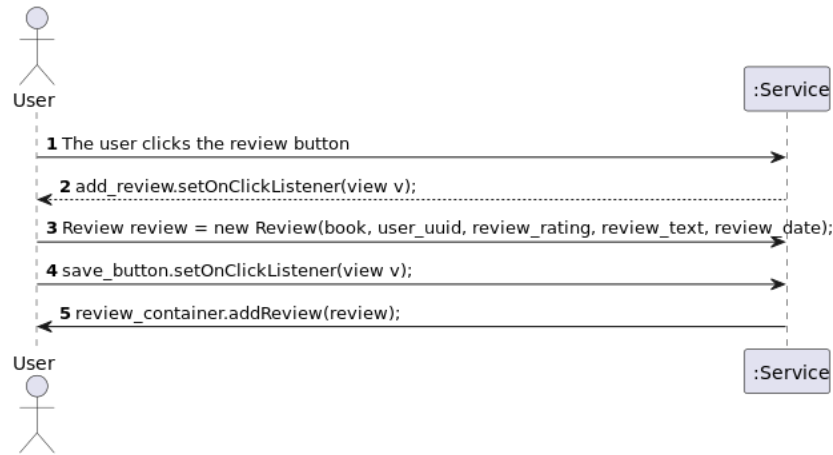
SD: UC3 - Add a review

Fig. 72: Book Screen UC3 Sequence Diagram

Use Case: See book metadata (Specification)

Actor	User
Use case name	See book metadata
Description	The user clicks on the bookinfo button to see the book metadata
Precondition	The user must be logged in
Post-condition	User now sees and knows the book metadata
Main flow	1. System displays the book screen 2. User clicks on the book info button 3. Systems shows that book metadata
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 33: Use Case: See book metadata

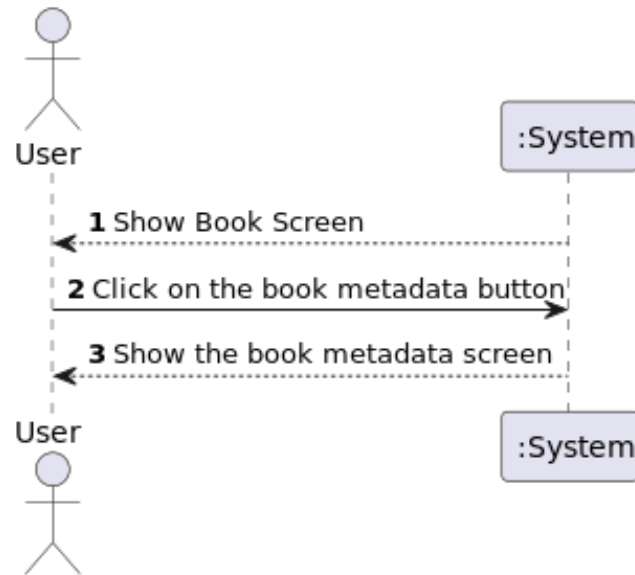
SSD: UC4 - See book metadata

Fig. 73: UC4 System Sequence Diagram

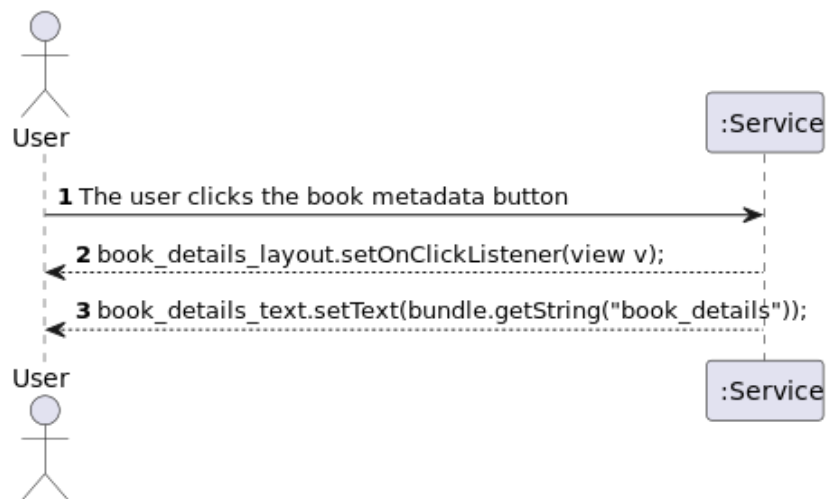
SD: UC4 - See book metadata

Fig. 74: Book Screen UC4 Sequence Diagram

Use Case: Share book (Specification)

Actor	User
Use case name	Share book
Description	The user clicks on the share book button to share the book
Precondition	The user must be logged in
Post-condition	User is send to book screen again
Main flow	1. System displays the book screen 2. User clicks on the share book button 3. User chooses where to send the book info (name and year) 4. System sends user to book screen again
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 34: Use Case: Share book

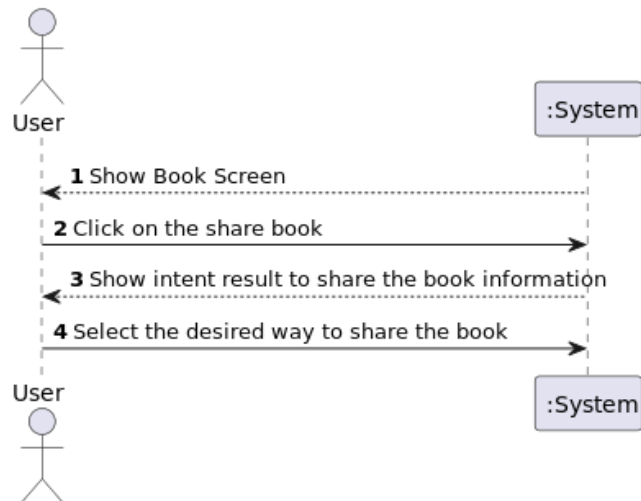
SSD: UC5 - Share book

Fig. 75: UC5 System Sequence Diagram

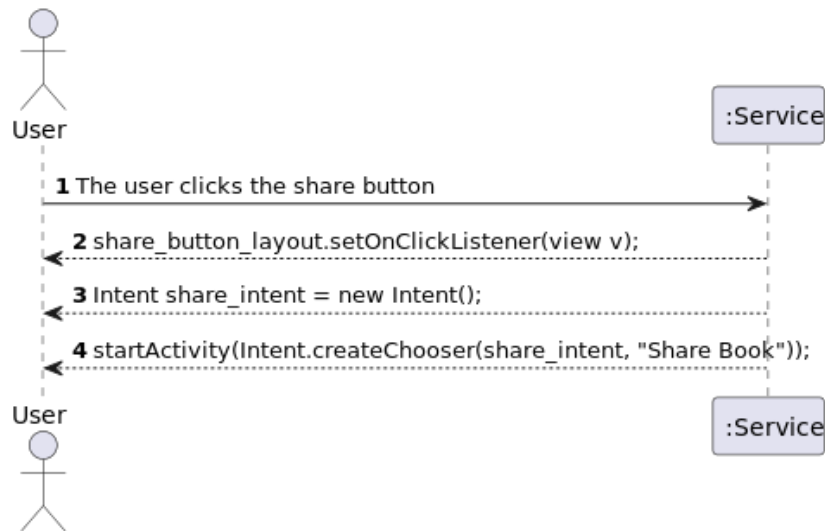
SD: UC5 - Share book

Fig. 76: Book Screen UC5 Sequence Diagram

Use Case: See book's rating history (Specification)

Actor	User
Use case name	See book's rating history
Description	The user sees the book's rating history
Precondition	The user must be logged in
Post-condition	User knows his rating history
Main flow	1. System displays the book screen 2. User sees his book rating history
Alternative path	None
Exceptions	If the user, at any point, wishes to cancel, the user can press the back button

Table 35: Use Case: See book's rating history

SSD: UC6 - See book's rating history

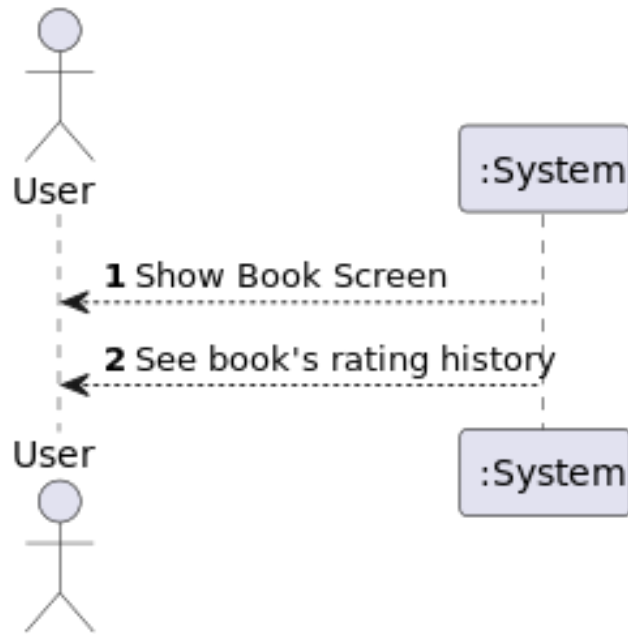


Fig. 77: UC6 System Sequence Diagram

SD: UC6 - See book's rating history

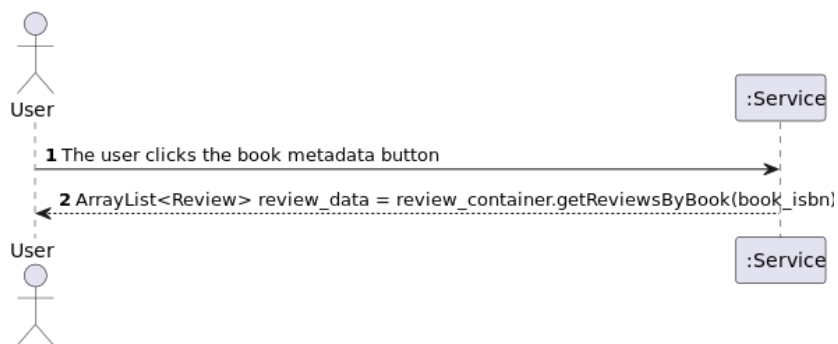


Fig. 78: Book Screen UC6 Sequence Diagram

6 Physical Architecture

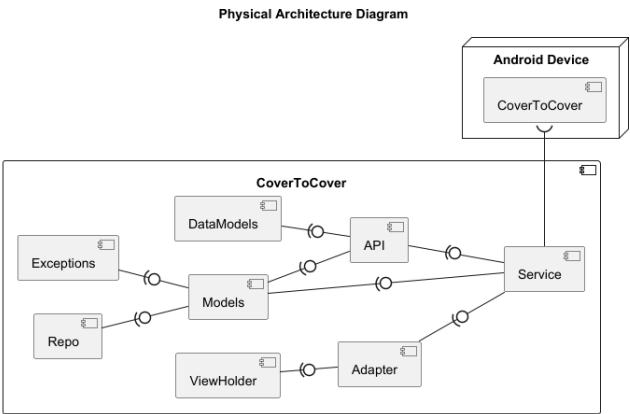


Fig. 79: CoverToCover Physical Architecture

7 Logical Architecture

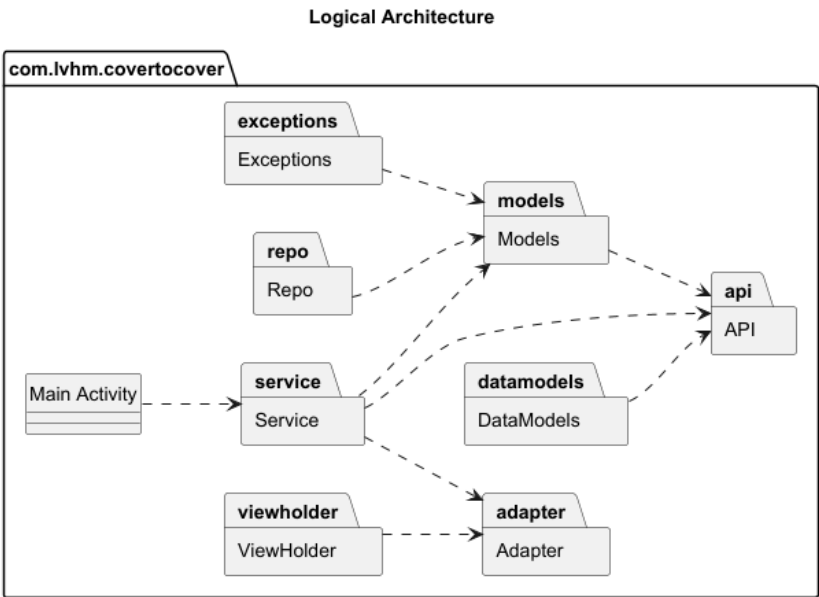


Fig. 80: CoverToCover Logical Architecture

8 Implementation

For the sake of not being too long, we will only show an example of how the code works. The example will be the search for a book in the API, using the camera.

8.1 Service

```
public class CameraScreen extends Fragment {
    private void startCamera() {
        if (!isAdded()) {
            return;
        }
        try {
            ProcessCameraProvider camera_provider =
                ↪ camera_provider_future.get();
            Preview preview = new Preview.Builder()
                .build();
            CameraSelector camera_selector = new
                ↪ CameraSelector.Builder()

                ↪ .requireLensFacing(CameraSelector.LENS_FACING_BACK)
                .build();
            ResolutionSelector resolution_selector = new
                ↪ ResolutionSelector.Builder()
                .setAspectRatioStrategy(
                    new AspectRatioStrategy(
                        AspectRatio.RATIO_16_9,

                        ↪ AspectRatioStrategy.FALLBACK_RULE_AUTO
                    )
                )
                .build();
            image_analysis = new ImageAnalysis.Builder()
                .setResolutionSelector(resolution_selector)
                .setBackpressureStrategy(ImageAnalysis
                    .STRATEGY_KEEP_ONLY_LATEST)
                .build();

            image_analysis.setAnalyzer(ContextCompat
                .getMainExecutor(requireContext()),
                this::scanBarcode);

            ↪ preview.setSurfaceProvider(preview_view.getSurfaceProvider());
            camera_provider.unbindAll();
            camera_provider.bindToLifecycle(this, camera_selector,
                ↪ preview, image_analysis);
```

```

    } catch (ExecutionException | InterruptedException error)
    ↪ {
        if (!isAdded()) {
            return;
        }
        String error_message = "Error initializing camera: " +
        ↪ error.getMessage();
        NotificationCentral.showNotification(requireContext(),
        ↪ error_message);
    }
}
@SuppressWarnings("UnsafeOptInUsageError")
private void scanBarcode(ImageProxy image_proxy) {
    Image media_image = image_proxy.getImage();
    if (media_image == null) {
        image_proxy.close();
        return;
    }
    BarcodeScannerOptions options = new
    ↪ BarcodeScannerOptions.Builder()
        .setBarcodeFormats(Barcode.FORMAT_EAN_13)
        .build();
    BarcodeScanner scanner =
    ↪ BarcodeScanning.getClient(options);
    processImageFeed(scanner, image_proxy, media_image);
}
private void processImageFeed(BarcodeScanner scanner,
    ↪ ImageProxy image_proxy, Image media_image) {
    InputImage image_to_scan =
    ↪ InputImage.fromMediaImage(media_image,
    ↪ image_proxy.getImageInfo().getRotationDegrees());
    scanner.process(image_to_scan)
        .addOnSuccessListener(barcodes -> {
            if (!isAdded()) {
                return;
            }
            if (barcodes.isEmpty()) {
                barcode_frame.updateBarcode(null);
            } else {
                Barcode barcode = barcodes.get(0);
                if (barcode.getBoundingBox() != null) {
                    barcode_frame
                        .updateBarcode(mapBarcodeFrame(barcode
                        .getBoundingBox(),
                        image_proxy));
                }
            }
        });
}

```

```

        }
        if (should_read_barcode) {
            should_read_barcode = false;
            String raw_value =
                ↪ barcode.getRawValue();
            getBookFromAPI(raw_value);
        }
    }
})
.addOnFailureListener(e -> {
    barcode_frame.updateBarcode(null);
    if (should_read_barcode) {
        should_read_barcode = false;
        if (isAdded()) {
            String error_message = "Error reading
                ↪ code: " + e.getMessage();
            NotificationCentral
                .showNotification(requireContext(),
                    ↪ error_message);
        }
    }
})
.addOnCompleteListener(task -> {
    image_proxy.close();
});
}

private void getBookFromAPI(String barcodeValue) {
    if (!isAdded()) {return;}
    BookAPIClient.getBookFromAPI(requireContext(),
        ↪ barcodeValue, new BookAPICallback() {
        @Override
        public void onBookFound(BookResponse.Item item, String
            ↪ isbn, Bundle response_bundle) {
            if (!isAdded()) {return;}
            navigateToBookScreen(item, isbn, response_bundle);
        }
        @Override
        public void onNoBookFound(String isbn) {
            if (!isAdded()) {return;}
            navigateToManualBookScreen(isbn);
        }
        @Override
        public void onAPIFailure(String errorMessage) {}
    });
}
}

```



```

private void navigateToBookScreen(BookResponse.Item item,
    ↪ String isbn, Bundle response_bundle) {
    Bundle book_details_bundle =
        ↪ item.getVolumeInfo().getBundle();
    book_details_bundle.putString("isbn", isbn);
    book_details_bundle.putBundle("response",
        ↪ response_bundle);
    if (getActivity() instanceof BookNavigationListener) {
        ((BookNavigationListener) getActivity())
            .navigateToBookScreenFromAPI(book_details_bundle);
    }
}

private void navigateToManualBookScreen(String isbn) {
    Fragment manual_book_screen_fragment = new
        ↪ ManualBookScreen();
    Bundle book_info = new Bundle();
    book_info.putString("isbn", isbn);
    manual_book_screen_fragment.setArguments(book_info);
    getParentFragmentManager()
        .beginTransaction()
        .replace(R.id.fragment_container,
            ↪ manual_book_screen_fragment)
        .addToBackStack(null)
        .commit();
}
}

```

8.2 API

```

public class BookAPIClient {
    private static Retrofit retrofit;
    private static final String BASE_URL =
        ↪ "https://www.googleapis.com/books/v1/";
    public static void getBookFromAPI(Context context, String
        ↪ barcode_value, BookAPICallback callback) {
        String query = "isbn:" + barcode_value.trim();
        BookAPIService book_api_service = getBookAPIService();
        Call<BookResponse> call =
            ↪ book_api_service.getBookByISBN(query);
        call.enqueue(new Callback<>() {
            @Override
            public void onResponse(@NonNull Call<BookResponse>
                ↪ call, @NonNull Response<BookResponse> response) {
                final Bundle book_details_bundle = new Bundle();
                if (response.isSuccessful() && response.body() !=
                    ↪ null && response.body().getItems() != null &&
                    ↪ !response.body().getItems().isEmpty()) {

```

```

        Gson gson = new Gson();
        book_details_bundle.putString("book_details",
        ↪ gson.toJson(response.body()));
        BookResponse.Item item =
        ↪ response.body().getItems().get(0);
        if (item != null && item.getVolumeInfo() !=
        ↪ null) {
            Toast.makeText(context, "Book found: " +
            ↪ barcode_value,
            ↪ Toast.LENGTH_SHORT).show();
            callback.onBookFound(item, barcode_value,
            ↪ book_details_bundle);
        } else {
            Toast.makeText(context, "Book found, but
            ↪ volume info is missing",
            ↪ Toast.LENGTH_SHORT).show();
            callback.onNoBookFound(barcode_value);
        }
    } else {
        Toast.makeText(context, "API call successful,
        ↪ but no book found",
        ↪ Toast.LENGTH_SHORT).show();
        callback.onNoBookFound(barcode_value);
    }
}

@Override
public void onFailure(@NonNull Call<BookResponse>
    ↪ call, @NonNull Throwable t) {
    Toast.makeText(context, "API call failed: " +
    ↪ t.getMessage(), Toast.LENGTH_SHORT).show();
    callback.onAPIFailure(t.getMessage());
}
}
});
}
}

```

9 Tests

As an example of the unit tests that were made. We show the unit test class for the Book class, as well as the model class.

```
public class Book implements Parcelable {
    @SerializedName("_id")
    private String _id;

    @Expose
    private String user_uuid;
    @Expose
    private String uuid;
    @Expose
    private String isbn;
    @Expose
    private String name;
    @Expose
    private ArrayList<String> author;
    @Expose
    private int year;
    @Expose
    private ArrayList<String> genre;
    @Expose
    private boolean read;
    @Expose
    private int page_count;
    @Expose
    private String cover_image64;
    @Expose(serialize = false, deserialize = false)
    private Bitmap cover_image;
    @Expose
    private boolean isWishlisted;
    @Expose
    private boolean onGoing;

    public Book() {
        this._id = "";
        this.user_uuid = "";
        this.uuid = UUID.randomUUID().toString();
        this.isbn = "Unknown";
        this.name = "Unknown";
        this.author = new ArrayList<>(List.of("Unknown"));
        this.year = 0;
        this.genre = new ArrayList<>(List.of("Unknown"));
        this.read = false;
        this.page_count = 0;
    }
}
```

```

        this.cover_image64 = null;
        this.cover_image = null;
        this.isWishlisted = false;
        this.onGoing = false;
    }
    public Book(String user_uuid, String isbn, String name,
        ↪ ArrayList<String> author,
            int year, ArrayList<String> genre, boolean read,
            ↪ int page_count,
                String cover_image64, Bitmap cover_image, boolean
                    ↪ isWishlisted, boolean onGoing) {
        this._id = "";
        this.user_uuid = user_uuid;
        this.uuid = UUID.randomUUID().toString();
        this.isbn = isbn;
        this.name = name;
        this.author = author;
        this.year = year;
        this.genre = genre;
        this.read = read;
        this.page_count = page_count;
        this.cover_image64 = cover_image64;
        this.cover_image = cover_image;
        this.isWishlisted = isWishlisted;
        this.onGoing = onGoing;
    }

    public String get_id() {
        return _id;
    }
    public void set_id(String id) {
        this._id = id;
    }
    public String getUserUUID() {
        return this.user_uuid;
    }
    public void setUserUUID(String user_uuid) {
        this.user_uuid = user_uuid;
    }
    public String getUUID() {
        return this.uuid;
    }
    public void setUUID(String uuid) {
        this.uuid = uuid;
    }
    public String getISBN() {

```

```
        return this.isbn;
    }
    public void setISBN(String isbn) {
        this.isbn = isbn;
    }
    public String getName() {
        return this.name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public ArrayList<String> getAuthor() {
        return this.author;
    }
    public void setAuthor(ArrayList<String> author) {
        this.author = author;
    }
    public void setAuthor(String authors) {
        this.author = (ArrayList<String>)
            ↪ Arrays.asList(authors.split(", "));
    }
    public int getYear() {
        return this.year;
    }
    public void setYear(int year) {
        this.year = year;
    }
    public ArrayList<String> getGenre() {
        return this.genre;
    }
    public void setGenre(ArrayList<String> genre) {
        this.genre = genre;
    }
    public boolean getRead() {
        return this.read;
    }
    public void setRead(boolean read) {
        this.read = read;
    }
    public int getPageCount() {
        return this.page_count;
    }
    public void setPageCount(int page_count) {
        this.page_count = page_count;
    }
    public String getCoverImageBase64() {
```

```

        return this.cover_image64;
    }

    public Bitmap getCoverImage() {
        String image_base64 = this.cover_image64;
        if (image_base64 != null) {
            try {
                byte[] byteArray = Base64.decode(image_base64,
                    ↪ Base64.DEFAULT);
                Bitmap bitmap =
                    ↪ BitmapFactory.decodeByteArray(byteArray, 0,
                    ↪ byteArray.length);
                if (bitmap != null) {
                    return bitmap;
                }
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
        return null;
    }

    public void setCoverImageBase64(Bitmap cover_image) {
        if(cover_image != null) {
            ByteArrayOutputStream baos = new
                ↪ ByteArrayOutputStream();
            cover_image.compress(Bitmap.CompressFormat.WEBP, 80,
                ↪ baos);
            byte[] byteArray = baos.toByteArray();
            this.cover_image64 = Base64.encodeToString(byteArray,
                ↪ Base64.DEFAULT);
        }
        else {
            this.cover_image64 = null;
        }
    }

    public void setCoverImage(Bitmap cover_image) {
        this.cover_image = cover_image;
    }

    public boolean getIsWishlisted() {
        return this.isWishlisted;
    }

    public void setIsWishlisted(boolean isWishlisted) {
        this.isWishlisted = isWishlisted;
    }

    public boolean getOnGoing() {
        return this.onGoing;
    }
}

```

```

        public void setOnGoing(boolean onGoing) {
            this.onGoing = onGoing;
        }
        ...
    }

@RunWith(RobolectricTestRunner.class)
public class BookTest {
    @Mock
    private Book book_get;
    @Mock
    private Book book_set;
    @Mock
    private Bitmap mock_bitmap;

    @Before
    public void setUp() {
        MockitoAnnotations.openMocks(this);
        book_get = new
        ↪ Book("1a2b3c4d-5e6f-7000-0000-000000000000",
            "9781408855652",
            "Harry Potter and the Philosopher's Stone",
            new ArrayList<>(Arrays.asList("J.K. Rowling")),
            1997,
            new ArrayList<>(Arrays.asList("Fantasy", "Young
            ↪ Adult")),
            false,
            223,
            "image_image_image",
            null,
            true,
            false
        );
        book_set = new Book();
    }

    @Test
    public void testGetUserUUID() {
        String expected_uuid =
        ↪ "1a2b3c4d-5e6f-7000-0000-000000000000";
        Assert.assertEquals(expected_uuid,
        ↪ book_get.getUserUUID());
    }

    @Test
    public void testSetUserUUID() {

```

```

        String expected_uuid =
            ↪ "1a2b3c4d-5e6f-7000-0000-000000000001";
        book_set.setUserUUID(expected_uuid);
        Assert.assertEquals(expected_uuid,
            ↪ book_set.getUserUUID());
    }

    @Test
    public void testGetISBN() {
        String expected_isbn = "9781408855652";
        Assert.assertEquals(expected_isbn, book_get.getISBN());
    }

    @Test
    public void testSetISBN() {
        String expected_isbn = "9781408855651";
        book_set.setISBN(expected_isbn);
        Assert.assertEquals(expected_isbn, book_set.getISBN());
    }

    @Test
    public void testGetTitle() {
        String expected_title = "Harry Potter and the
            ↪ Philosopher's Stone";
        Assert.assertEquals(expected_title, book_get.getName());
    }

    @Test
    public void testSetTitle() {
        String expected_title = "Harry Potter and the Chamber of
            ↪ Secrets";
        book_set.setName(expected_title);
        Assert.assertEquals(expected_title, book_set.getName());
    }

    @Test
    public void testGetAuthors() {
        ArrayList<String> expected_authors = new
            ↪ ArrayList<>(Arrays.asList("J.K. Rowling"));
        Assert.assertEquals(expected_authors,
            ↪ book_get.getAuthor());
    }

    @Test
    public void testSetAuthors() {
        ArrayList<String> expected_authors = new
            ↪ ArrayList<>(Arrays.asList("J.K. Rowling", "J.R.R.
            ↪ Tolkien"));
        book_set.setAuthor(expected_authors);
        Assert.assertEquals(expected_authors,
            ↪ book_set.getAuthor());
    }
}

```



```

@Test
public void testGetYear() {
    int expected_year = 1997;
    Assert.assertEquals(expected_year, book_get.getYear());
}

@Test
public void testSetYear() {
    int expected_year = 1998;
    book_set.setYear(expected_year);
    Assert.assertEquals(expected_year, book_set.getYear());
}

@Test
public void testGetGenres() {
    ArrayList<String> expected_genres = new
        ArrayList<>(Arrays.asList("Fantasy", "Young Adult"));
    Assert.assertEquals(expected_genres, book_get.getGenre());
}

@Test
public void testSetGenres() {
    ArrayList<String> expected_genres = new
        ArrayList<>(Arrays.asList("Fantasy", "Young Adult",
        "Science Fiction"));
    book_set.setGenre(expected_genres);
    Assert.assertEquals(expected_genres, book_set.getGenre());
}

@Test
public void testGetRead() {
    Assert.assertFalse(book_get.getRead());
}

@Test
public void testSetRead() {
    book_set.setRead(true);
    Assert.assertTrue(book_set.getRead());
}

@Test
public void testGetPages() {
    int expected_pages = 223;
    Assert.assertEquals(expected_pages,
        book_get.getPageCount());
}

@Test
public void testSetPages() {
    int expected_pages = 224;
    book_set.setPageCount(expected_pages);
    Assert.assertEquals(expected_pages,
        book_set.getPageCount());
}

```

```

    }
    @Test
    public void testGetImage64() {
        String expected_image64 = "image_image_image";
        Assert.assertEquals(expected_image64,
            ↪ book_get.getCoverImageBase64());
    }
    @Test
    public void testSetImage64() {
        Bitmap bitmap = Bitmap.createBitmap(1, 1,
            ↪ Bitmap.Config.ARGB_8888);
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        bitmap.compress(Bitmap.CompressFormat.WEBP, 80, baos);
        String expected_image64 =
            ↪ Base64.encodeToString(baos.toByteArray(),
            ↪ Base64.DEFAULT);
        book_set.setCoverImageBase64(bitmap);
        Assert.assertEquals(expected_image64,
            ↪ book_set.getCoverImageBase64());
    }
    @Test
    public void testGetCoverImage() {
        Assert.assertNull(book_get.getCoverImage());
    }
    @Test
    public void testGetWishlisted() {
        Assert.assertTrue(book_get.getIsWishlisted());
    }
    @Test
    public void testSetWishlisted() {
        book_set.setIsWishlisted(false);
        Assert.assertFalse(book_set.getIsWishlisted());
    }
    @Test
    public void testGetOnGoing() {
        Assert.assertFalse(book_get.getOnGoing());
    }
    @Test
    public void testSetOnGoing() {
        book_set.setOnGoing(false);
        Assert.assertFalse(book_set.getOnGoing());
    }
}
}

```

10 Conclusion

CoverToCover successfully delivers on its promise to create a frictionless and private environment for literary expression. By eliminating the stress and anxiety often associated with social pressure and online sharing, and crucially, guaranteeing that no one can judge the user's opinion, the app fundamentally redefines the way readers connect with and review their books. It is the definitive and trustworthy companion for every reading journey.